

A cell-centered AMR-ALE framework for 3D multi-material hydrodynamics.

Part I: Lagrangian and indirect Euler AMR algorithms

A. Colaïtis^{1,2}, S. Guisset³, J. Breil³

Abstract. Many applications of physics and engineering involve wide ranges of time and spatial scales. The numerical simulation of localized small scales such as shock waves and material interfaces requires a large number of computational cells in these regions. For these applications, Lagrangian and Arbitrary-Lagrangian-Eulerian (ALE) related methods are engaging since the moving mesh feature naturally brings mesh cells on shock discontinuities and material interfaces are carefully captured. In addition, Adaptive-Mesh-Refinement (AMR) strategies aim to optimize computational resources by concentrating finer mesh cells only in areas of interest while using coarser cells elsewhere. A key but challenging AMR requirement consists in efficiently distributing the computational effort to achieve high accuracy without the prohibitive computational costs associated with uniformly fine grids. In this document, the coupling of the *p4est* AMR library with a cell-centered Lagrangian scheme is presented with the goal to perform reliable 3D Lagrangian-AMR and indirect Euler-AMR multi-material simulations. In particular, it is shown that starting from a 3D indirect ALE code, the memory management and load balancing requirements can be delegated to an external library (here the *p4est* library) to unlock ALE-AMR capabilities. First, we present a strategy to transcribe the octant-based connectivity of the 3D AMR framework with that of an unstructured mesh of polygonal cells used in Lagrangian hydrodynamics. Then, we show how refinement and coarsening operations must be adapted to the particular Lagrangian framework to ensure the conservation of volume during those steps. Finally, several numerical test cases are presented that demonstrate the capabilities of the Lagrangian-AMR and indirect Euler-AMR algorithms.

Key words. Adaptive-Mesh-refinement (AMR), Arbitrary-Lagrangian-Eulerian (ALE), 3D cell-centered Lagrangian numerical scheme, multimaterial flows.

1. Introduction

Multi-material flows are omnipresent in nature and industry. Understanding complex interactions between material phases and predicting the behavior of such flows is of paramount importance in various engineering applications. However, the inherent complexity of multimaterial flows, which may involve material interface dynamics, strong shock waves, mass and heat transfer, or phase transitions, makes the derivation of analytical solutions challenging or even impossible in most cases. As a consequence, numerical simulation has become an indispensable tool for studying multi-material systems. In practice, material interfaces are often treated as discontinuities because their widths are largely smaller than the macroscopic length scales of interest.

Over the years, the numerical simulation of these material interfaces has been largely investigated and numerous models and numerical approaches have been proposed. Among them, two main approaches have emerged. The first category is known in the literature as diffuse interface methods. This approach considers

¹Laboratory for Laser Energetics, 250 East River Rd, Rochester, NY 14623-1212

²Corresponding author: acol@le.rochester.edu

³CEA/CESTA, 15 Avenue des Sablières CS 60001, 33116 Le Barp cedex, France

interface areas in which the materials are allowed to be artificially mixed. Here, interfaces are considered as diffuse zones. This strategy has recently become popular since the geometrical technicalities are alleviated. In addition, the development of anti-diffusive methods, combined with an increased computational power, contributed to reducing the numerical smearing of material interfaces [1, 2, 3]. The second category is referred to as sharp interface methods. In this case, the materials are seen as separated entities, and the interfaces correspond to the boundary between them. Here, no mixing between materials is authorized. This approach is particularly popular in the community since there is no numerical diffusion of the material interfaces. The multimaterial modeling strategy used in this study is based on the sharp interface method and a standard closure of equal compressibility (equal strain assumption) [4]. This last hypothesis assumes that the volume fractions of the material remain constant during the Lagrangian phase, such that all materials are compressed or expanded at the same rate. This assumption is attractive because of its obvious simplicity and may be relevant enough for some multiphase applications [4]. However, when working with very contrasted materials (e.g. air and water), strong non-physical pressure decoupling may occur in mixed cells. This artificial separation is an issue in some multi-material applications since it may result in a crash of the code. Other closures have then been proposed in the literature to handle mixed-cells filled by materials with strong compressibility differences. We refer to [5] in which these aspects have been investigated.

The numerical strategy used in this study dwells in the category of *Arbitrary-Lagrangian-Eulerian* (ALE) methods [6]. ALE strategies are usually divided in two groups. The first is designated in the scientific literature as *direct ALE* and deals with the numerical resolution of the fluid equations by working with a moving arbitrary mesh. In this case, the advection terms are directly incorporated in the overall numerical resolution. The second strategy is often referred to as the *Lagrangian plus Remapping* or *indirect ALE* methods and is the one used in the present study. In this case, the flow equations are first solved within the Lagrangian formalism (i.e., in the framework of the fluid) in which the mesh follows the fluid velocity. The Lagrangian framework is particularly attractive since multi-material interfaces are captured exactly. In addition, it enables very natural multiphysics couplings (such as radiation terms), robust handling of source terms, and more straightforward material thermodynamic consistency. On the down side, as the simulated flow may encounter large deformations, the mesh quality can be strongly impacted, resulting in undesirable mesh entanglement. Here an effective mesh regularization procedure may be required to modify the node positions in order to improve the global mesh quality. Obviously, the numerical solutions computed through the Lagrangian phase should be carefully remapped onto the regular mesh. This remapping step is sensitive when dealing with multi-material problems and must be attentively addressed. The *Lagrangian plus Remapping* strategy is notably efficient, since the remapping step onto the regular mesh provides the robustness of the overall numerical method while preserving some pleasant features of the Lagrangian step.

Various numerical strategies have been developed to effectively solve the hydrodynamic equations written in the Lagrangian formalism. A complete review is not the purpose, so we simply point out the most common ones. Among the most famous are the *staggered* schemes for which all kinematic variables (node velocity and position) are evaluated at the nodes of the mesh, while the thermodynamic quantities (usually density and internal energy) are computed at the center of the cells. From the early work [7, 8] various extensions have been proposed over the years, making them commonly used today in very diverse application fields. A second category of well-known numerical methods is referred as *cell-centered* schemes. Following the early work of Godunov [9], various extensions have been proposed for the Lagrangian formalism. All the quantities (density, momentum, internal or total energy) are computed inside the cell, while the nodes positions are computed from the nodes velocities obtained from a nodal solver. It should also be noted that finite element methods can be used to solve the Lagrangian hydrodynamics equations [10, 11].

Following the indirect ALE methodology, the regularization step is intended to increase overall mesh quality while trying to maintain some attractive Lagrangian features [12]. In the present work, the mesh regularization procedure is not detailed. Indeed, this step is sensitive and challenging in this 3D ALE-AMR framework, and left to the second part of this study [13]. The 3D multi-material remapping step used in this document is based on a face-sweeping technique which is used extensively. Introduced in [14], this popular strategy consists in computing the volume integral of a new cell (cell obtained from the regularization algorithm) starting from the integral of the old cell (cell obtained from the Lagrangian step) plus the contributions of the swept regions by the displacement of the cell faces. This sweeping approach is

often chosen over the cell-intersection-based method which is expensive in a 3D setting.

Many real-life engineering applications involve large ranges of time and spatial scales. The numerical simulation of localized small scales such as shock waves or material interfaces requires a large number of computational cells in these regions. For these applications, Lagrangian and ALE related methods are engaging since the moving mesh feature naturally brings mesh cells on shock discontinuities and material interfaces are carefully captured. However, despite their natural mesh adaptive features, they can not compete with truly dynamic *Adaptive-Mesh-Refinement* (AMR) strategies. The aim of AMR algorithms is to optimize computational resources by concentrating finer mesh cells only in areas of interest while using coarser cells elsewhere. Based on user-defined criteria, often related to error estimates or local solution features, the mesh grid is automatically refined or coarsened. A key but challenging requirement to achieve efficient AMR algorithms is to efficiently distribute the computational effort. In that case, AMR simulations achieve high accuracy without the prohibitive computational costs associated with uniformly fine grids. The mesh refinement approach is generally implemented through hierarchical grid structures, where finer grids are embedded within coarser grids, with the goal of achieving seamless resolution transitions. Since the pioneer works [15, 16, 17, 18] in which AMR algorithms were studied for the numerical simulation of hyperbolic partial differential equations, AMR concepts have been widely extended. Over the years, various physical simulations have been undertaken using the AMR technology. However, despite the need of AMR strategies for many physical applications, their uses remain complex. The first difficulty comes from the chosen numerical method, which can be entangled in a convoluted way with the AMR strategy. Even worse, the domain decomposition issue associated to the load balancing requirement for parallel computing in both shared and distributed memory architectures is particularly sensitive. This critical point demands the use of expensive methods based on key ideas from graph partitioning theory [19]. To address these challenging issues, external AMR libraries have been developed to ease the connection and the use of AMR techniques with existing simulation codes. Consequently, various AMR codes have emerged and can now be found in the literature. We do not intend to review all AMR libraries or AMR based code and simply refer to [20] and the references therein for more details.

In the present work, the *p4est* AMR library [21] has been selected. The main motivation behind this choice comes mainly from the variety of advanced simulation codes which leverage the *p4est* library for its powerful adaptive mesh refinement capabilities, particularly in large-scale and complex simulations [22, 23, 24]. The *p4est* library is designed to scale efficiently on large computer architectures. It excels at parallel load balancing, ensuring even distribution of the computational workload across processors, and oversees dynamic refinement-coarsening for quadtree(2D)/octree(3D)-based grids. *p4est* heavily relies on the use of linear storage given by a space-filling curve. Space-filling curves enable an efficient way of partitioning data for load balancing in distributed architecture. *p4est* also uses these space-filling curve notions for its data-memory layout management. This point should be underlined, since space-filling curves-based algorithms enjoy very compelling performances because of their efficient memory access. This feature is due to a link between the contiguity of the space filling curve with the contiguity of the N-dimensional Cartesian space mesh used for numerical simulations, see [21] and the references therein. Finally, the *p4est* library enables a great geometric flexibility naturally suited for a wide range of physical problems involving complex domains. *p4est* relies on the notion of *forest of trees* which enables the use of several deformed but conforming and adjacent meshes (each mesh corresponds to a tree). Although this strategy does not have the same flexibility as truly unstructured meshes, it enables to represent very complex geometries.

Lagrangian simulations naturally tend to concentrate the mesh cells in regions where shock and contact discontinuities are entailed while leaving some areas under-resolved such as those which involve rarefactions. This drawback may be greatly alleviated thanks to an efficient AMR strategy and is one of the motivations for coupling AMR to ALE. However, it should be noted that only a few ALE-AMR published works are available in the scientific literature. As a matter of fact, efficiently combining an ALE approach with AMR technologies is a challenging topic. Among the main difficulties, one can cite the complexity to link the AMR technology which traditionally uses Cartesian meshes with the Lagrangian formalism in which the mesh moves with the flows. In addition, standard Lagrangian schemes require the evaluation of different quantities at the nodes. Staggered Lagrangian schemes use nodal velocities and positions, while cell-centered Lagrangian ones rely on nodal-based solvers. Accessing nodal-related quantities is intricate, as it is not

adapted to the AMR technology that was initially developed and suited for cell-centered discretizations. In addition, the extension of standard rezoning strategies to the AMR formalism is not straightforward and needs to be carefully investigated. Among the few ALE-AMR works encountered in the literature, we cite [25] in which a staggered-grid ALE strategy is coupled in the 2D/3D framework with a structured local adaptive mesh refinement method. This is notably the basis of LLNL's ARES multiphysics code [26], which is, however, largely unpublished. We also mention [27], in which an ALE-AMR is presented for the 2D/3D simulations of free surface flows. In [28], a direct ALE-AMR method is presented for the simulation of shock hydrodynamics on 3D unstructured grids. Hierarchical refinement of tetrahedral cells is used to achieve speed-ups over uniform fine mesh solutions of $5 - 10\times$ for a given level of error. The two last mentioned papers use tetrahedral cells and do not address the multi-material aspects.

This is a two part paper. The first part presented focuses on efficiently coupling the *p4est*-AMR library with the 3D cell-centered Lagrangian algorithms. The second part introduces the adaptation of the ALE rezoning strategy (including disentangling of ALE-AMR meshes) to the Lagrangian-AMR scheme presented in Part I, allowing one to conduct fully ALE-AMR calculations. It is presented in [13]. In our opinion, the originality of this work stems from multiple factors. Firstly, to our knowledge it is the first time an AMR strategy is coupled with a cell-centered Lagrangian scheme to perform reliable 3D indirect ALE-AMR multi-material simulations. As explained above, this coupling is challenging and sensitive, but leads the way toward fast and accurate indirect ALE simulations of 3D multi-material physical problems. Secondly, even if this requires a heavy load of work, it is shown that starting from a 3D indirect ALE code, the memory management and load balancing requirements can be delegated to an external library (in our case the *p4est* library) to unlock ALE-AMR capabilities. Third, the management of Lagrangian mesh deformation combined with the refinement/coarsening AMR steps is clearly detailed. Finally, different numerical test cases are presented in depth, demonstrating the capabilities of the ALE-AMR strategy.

The document is organized as follows. In section 2, the basic elements of the 3D cell-centered Lagrangian scheme used in the rest of the paper are recalled. Then, we present a multi-material ALE remapping algorithm in section 3, which will be required for the AMR algorithm. The formalism of the parallel forest of octrees is introduced in section 4 as the basis for the Lagrangian mesh. We then introduce the algorithms for cell-centered AMR on Lagrangian meshes in section 5. The volume conservation requirement is presented in detail, as well as the AMR remapping on the preprocessed mesh. The refinement/coarsening AMR criteria are also presented. Finally, several numerical test cases are presented in section 6 to demonstrate the potential of the Lagrangian-AMR and indirect Euler-AMR algorithms.

2. Background on 3D cell-centered Lagrangian schemes

2.1. Definitions and notations

The notations used in the following sections are similar to the ones introduced in [29] and are briefly summarized. The computational domain is denoted $\omega(t)$. It is paved with non-overlapping polyhedrons denoted ω_c such that $\omega(t) = \bigcup_c \omega_c$. In this 3D context, the faces may not be planar, therefore the definitions of outward normal and area is not straightforward. In [29], cells are subdivided by adding one point p_f^* for each face. This choice enables us to prove the GCL property for the 3D Lagrangian phase (discrete compatibility relation between mass conservation and mesh geometry) and enforces correct discrete symmetry preservation of the flow. Concerning the notation introduced in [29], the index c stands for a cell ω_c , f refers to a face and p to a node. The triangles created by the face splitting are denoted t_r . For completeness, we recall all the notations used:

- $\mathcal{P}(c)$ is the set of nodes incident on cell c (without the nodes p_f^*),
- $\mathcal{P}(f)$ is the set of nodes incident on face f (without the node p_f^*),
- $\mathcal{P}(p)$ is the set of nodes incident on node p through an edge,
- $\mathcal{P}(c)_o$ is the part of $\mathcal{P}(c)$ that is also a corner node in c

- $\mathcal{P}(f)_o$ is the part of $\mathcal{P}(f)$ that is also a corner node in any cell incident on f
- $\mathcal{F}(c, p)$ is the set of faces of cell c and sharing point p ,
- $\mathcal{C}(p)$ is the set of cells c' sharing node p ,
- $\mathcal{T}(c)$ is the set of all the triangles t_r resulting from the splitting of the faces of cell c ,
- $\mathcal{T}(c, f)$ is the set of triangles t_r resulting from the splitting of the face f of cell c ,
- $\mathcal{T}(c, f, p)$ is the set of triangles t_r resulting from the splitting of the face f of cell c and sharing point p .

2.2. Cell-centered numerical strategy for 3D semi-Lagrangian hydrodynamics

This section starts with a brief recall of the cell-centered numerical discretization. This part is taken directly from [30, 29] and is concisely summarized. It constitutes the building block on which the 3D ALE-AMR strategy presented in the next sections is based. The Lagrangian step considered is taken directly from [29]. Concerning the cell variables, the approach is that of a finite-volume scheme. More precisely, one defines the averaged quantities called mass, velocity and total energy over the cell c , by

$$m_c = \int_{\omega_c} \rho d\mathbf{x}, \quad \mathbf{V} = \frac{1}{m_c} \int_{\omega_c} \rho \mathbf{V} d\mathbf{x}, \quad E_c = \frac{1}{m_c} \int_{\omega_c} \rho E d\mathbf{x}.$$

In this Lagrangian framework, the mass conservation equation imposes the cell mass m_c to be constant. From [29], semi-discrete momentum and total energy conservation equations reads

$$\begin{aligned} m_c \frac{d\mathbf{V}_c}{dt} + \sum_{p \in \mathcal{P}(c)} \sum_{f \in \mathcal{F}(c, p)} S_{pf} P_{cfp} \mathbf{n}_{pf} &= \mathbf{0}, \\ m_c \frac{dE_c}{dt} + \sum_{p \in \mathcal{P}(c)} \sum_{f \in \mathcal{F}(c, p)} S_{pf} P_{cfp} \mathbf{V}_p \cdot \mathbf{n}_{pf} &= 0, \end{aligned} \quad (1)$$

where the index c denotes that the quantity has been mass averaged over the considered cell. The semi-discrete trajectory equation simplifies into

$$\frac{d\mathbf{x}_p}{dt} = \mathbf{V}_p. \quad (2)$$

At this point, the nodal fluxes P_{cfp} and $\mathbf{V}_p$ are the remaining unknowns to be determined. A sufficient condition to ensure a positive entropy production considers the pressure jumps may be written in terms of the velocity jump

$$P_{cfp} - P_c = Z_c (\mathbf{V}_c - \mathbf{V}_p) \cdot \mathbf{n}_{pf}, \quad (3)$$

where $Z_c = (\rho a)_c$ defines the acoustic impedance inside cell c and $a = \sqrt{(dP/d\rho)_\eta}$ the speed of sound. In addition, discrete total momentum and energy conservation are obtained by defining the nodal velocity $\mathbf{V}_p$ solution of the following set of equations

$$\mathbb{M}_p \mathbf{V}_p = \mathbf{B}, \quad (4)$$

where

$$\begin{aligned} \mathbb{M}_p &= \sum_{c \in \mathcal{C}(p)} \sum_{f \in \mathcal{F}(c, p)} S_{pf} Z_c (\mathbf{n}_{pf} \otimes \mathbf{n}_{pf}), \\ \mathbf{B} &= \sum_{c \in \mathcal{C}(p)} \sum_{f \in \mathcal{F}(c, p)} S_{pf} \left[P_c \mathbf{n}_{pf} + Z_c (\mathbf{n}_{pf} \otimes \mathbf{n}_{pf}) \mathbf{V}_c \right]. \end{aligned}$$

The matrix $\mathbb{M}_p$ is positive definite and thus invertible, therefore the computation of the nodal velocity is straightforward. For completeness, we recall the definition of the face area vector $S_{pf}\mathbf{n}_{pf}$

$$S_{pf}\mathbf{n}_{pf} = \frac{1}{3} \left(\sum_{t_r \in \mathcal{T}(c,f,p)} S_{t_r}\mathbf{n}_{t_r} + \sum_{t_r \in \mathcal{T}(c,f)} \frac{1}{N_{p,f}} S_{t_r}\mathbf{n}_{t_r} \right), \quad (5)$$

and refers to [29] for its importance in enforcing the GCL condition. The $S_{pf}\mathbf{n}_{pf}$ term is called the face area vector. It may be seen as the contribution of the face f to the corner area vector where the corner area vector writes $\mathbf{n}_p = \sum_{f \in \mathcal{F}(c,p)} S_{pf}\mathbf{n}_{pf}$.

2.3. second-order extension

The EUCCLHYD scheme is extended at second-order in space using a MUSCL procedure (Monotonic Upstream-Centered Scheme for Conservation Laws) [31, 32]. Our unstructured AMR framework as described in Sec. 5 leads to a variety of cell neighborhood types. A same mesh can combine regions of regular connectivity (e.g. structured hexahedral mesh) with highly unstructured and nonconformal cells (e.g. cells with up to 24 faces whose neighbors have various refinement depth and/or nonconformal hanging nodes). For the second-order extension of the scheme, we need a slope-limitation procedure that performs well for a large variety of configuration. Notably, we require a procedure that preserves 1D symmetry of flows while staying as close to second-order as possible. Ideally, we would prefer a strategy that is applicable to all our flow configurations, i.e. 1D to 3D, planar to spherical, using the same limitation procedure.

Ref. [30] proposed two main strategies called Symmetry-Preserving (SP) and Multi-dimensional Minmod (MM) limiters. In both cases, the limitation is applied to the material pressure and velocity vector. The SP limiter slope-limits cell gradients through a Barth-Jespersen procedure whose extrema are computed from the nodal-neighborhood of the cell, and the second-order reconstruction is evaluated at the nodes of the cell. The MM limiter follows a more complicated procedure that aims to reproduce the classical minmod limiter in 1D and to filter components that are not aligned with the main flow. As such, the MM limitation procedure starts by computing nodal gradients, then projecting these gradients onto the cell velocity vector, and finally applying the minmod function to these projections to obtain the magnitude of the cell gradient, the direction of which is taken to be that of the flow velocity vector. Applications of our 3D hydrodynamic scheme to classical hydrodynamics instability test cases have shown that the MM limitation procedure is highly diffusive and gives incorrect growth-rates compared to the literature. We also found the SP limiter as formulated in Ref. [30] to be too diffusive, although not as much as the MM limiter.

In the present work, we use a slightly modified version of the SP limiter, denoted SP-V, which gives a correct growth rate for classical hydrodynamic instability growth (see part II of this work). This limiter is used in all the tests presented in this paper and in part II. It is found to perform well overall, but shows some limits for the spherical implosion test case of Part II, where defects are amplified along a triple-point-like topological junction between mesh blocks. Although accuracy in that case is still acceptable, and it does not hinder calculations in presence of pre-imposed perturbations, we note that further work could be done on slope limitation strategies. For clarity, we now detail our limitation procedure. The procedure starts by the computation of discrete cell gradients constructed from a discrete version of the Green formula (see [30])

$$\begin{aligned} (\nabla \hat{P})_c &= \frac{1}{v_c} \sum_{p \in \mathcal{P}(c)} \sum_{f \in \mathcal{F}(c,p)} S_{pf} P_{cfp} \mathbf{n}_{pf}, \\ (\nabla \hat{\mathbf{V}})_c &= \frac{1}{v_c} \sum_{p \in \mathcal{P}(c)} \sum_{f \in \mathcal{F}(c,p)} S_{pf} \mathbf{V}_p \otimes \mathbf{n}_{pf}, \end{aligned} \quad (6)$$

where the hat symbol denotes that these gradients have not been slope-limited and it should be noted that $(\nabla \hat{\mathbf{V}})_c$ is a 3×3 tensor. The unlimited gradients are slope-limited by applying a scalar factor to their amplitude, which is expressed as $(\nabla \psi)_c = \chi_c(\psi) (\nabla \hat{\psi})_c$ where $\chi_c(\psi) \in [0, 1]$ is a limitation function applied

to variable ψ (here ψ represents a scalar or vector). The limitation coefficient χ_c is then constructed as follows

$$\begin{aligned}\chi_c(\psi) &= \text{BJ} \left[\left\{ \psi_{c' \in \mathcal{C}(\mathcal{P}(c))} \right\}; \left\{ \psi_c + (\mathbf{r}_p - \mathbf{r}_c) \cdot (\nabla \hat{\psi})_c, \forall p \in \mathcal{P}(c) \right\}; \psi_c \right], \\ \chi_c(\mathbf{V}) &= \text{BJ} \left[\left\{ \mathbf{V}_{c' \in \mathcal{C}(\mathcal{P}(c))} \cdot \frac{\mathbf{V}_c}{|\mathbf{V}_c|} \right\}; \left\{ (\mathbf{V}_c + (\nabla \hat{\mathbf{V}})_c (\mathbf{r}_p - \mathbf{r}_c)) \cdot \frac{\mathbf{V}_c}{|\mathbf{V}_c|}, \forall p \in \mathcal{P}(c) \right\}; |\mathbf{V}_c| \right],\end{aligned}\quad (7)$$

where $\text{BJ}(\{a_i\}; \{b_j\}, d)$ denotes the Barth-Jespersen limitation function with the minima/maxima computed on the set of control values a and applied to the set of extrapolated values b , with respect to a reference value d

$$\begin{aligned}BJ(\{a_i\}; \{b_j\}, d) &= \min_j(\gamma_j) \\ \gamma_j &= \min \left(\frac{\max_i(a_i) - d}{b_j - d}, 1 \right) && \text{if } b_j - d > 0, \\ \gamma_j &= \max \left(\frac{\min_i(a_i) - d}{b_j - d}, 1 \right) && \text{if } b_j - d < 0, \\ \gamma_j &= 1 \text{ otherwise.}\end{aligned}\quad (8)$$

Note the following differences in our SP-V limiter compared to [30]:

- The stencil b_j taken for the extrapolated values is the position of the nodes of the cells, and not the centroids of the neighboring cells. This approach is less diffusive (the slope is less limited) and allows us to recover the correct growth rate in Rayleigh-Taylor single-mode growth rate calculations.
- For the velocity limitation, the computation of the control values a_i is made by projecting the velocities to that of the current cell c , contrary to the direction of the pressure gradient in [30]. This modification was found to be necessary for cases where additional source terms in the energy equation (from physics packages) significantly modify the pressure field and its gradient.

2.4. Multimaterial aspects

2.4.1. Multi-phase approximation

We start by recalling the strategy presented in [4]. Consider a mixture made of an arbitrary number of materials, the volume fraction of material k is denoted α^k and the corresponding density ρ^k . For a volume V in which a mass m^k of a material k occupies a volume V^k , the material volume fraction and material density are defined by

$$\alpha^k = \frac{V^k}{V}, \quad \rho^k = \frac{m^k}{V^k}.$$

Consequently, the total density of the mixture denoted ρ is defined by

$$\rho = \frac{m}{V} = \sum_k \alpha^k \rho^k.$$

Specific internal energy for material k is denoted ε^k and the corresponding internal energy for the mixture e is defined as follows

$$m\varepsilon = \sum_k m^k \varepsilon^k.$$

Each material k in the mixture is assigned a pressure p^k and entropy η^k . Finally, ρ^k , ε^k and p^k are related to each other by an equation of state $p^k(\rho^k, \varepsilon^k)$.

2.4.2. Lagrangian step: equal-strain assumption

In the present work, we restrict ourselves to the equal-strain assumption. We mention that other closure relations exist and have been studied in a similar framework [5]. The equal-strain assumption assumes that all material behaves following the same volume deformation

$$\frac{1}{\rho} \frac{d}{dt} \rho = \frac{1}{\rho^k} \frac{d}{dt} \rho^k = \nabla \cdot \mathbf{V}.$$

As detailed in [5], this assumption leads to

$$\frac{d}{dt} \alpha^k = 0.$$

The material density and internal evolution equations may then be simplified into

$$\alpha^k \rho^k \frac{d}{dt} \left(\frac{1}{\rho^k} \right) = \alpha^k \nabla \cdot \mathbf{V}, \quad \alpha^k \rho^k \frac{d}{dt} \varepsilon^k = -\alpha^k p^k \nabla \cdot \mathbf{V},$$

and the total pressure involved in the momentum evolution equation becomes $p = \sum_k \alpha^k p^k$. As pointed out in [5], this model is hyperbolic with the speed of sound a of the mixture given by

$$a^2 = \sum_k \frac{m^k}{m} (a^k)^2.$$

3. Second-order multimaterial remapping

In this section, we present our second-order multimaterial remapping method based on a 3D swept-face algorithm combining material interface reconstruction, surrogate prisms and flux corrected remapping.

3.1. Computation of material polytopes

The materials in cells are assumed to be separated by a planar interface Π^k that represents the position of the material interface k (Piece-Wise Linear Construction approach - PLIC [33]). This interface is characterized by a normal vector and a point. The normal vector is the direction of the volume fraction gradient $-\nabla \alpha^k$. With this definition, the portion of the cell occupied by material k is *below* the plane, which we denote by half-space Π^{k-} . This is illustrated on Fig. 1. The intersection of the cell polyhedra ω_c with this material plane Π_c^{k-} defines the material polytope ω_c^k of volume $V(\omega_c^k)$. The plane position is constrained by finding a point within the plane such that the volume of intersection between the cell c and the half-space Π_c^{k-} corresponds to $V(\omega_c^k) \equiv V_c^k$. The polygon volume is by construction a monotone function of the plane position along the normal between the two extrema positions d_0 and d_1 of the plane such that $\alpha_c^k = 0$ and $\alpha_c^k = 1$, respectively (see Fig. 1). Using the splitting of the faces of the cell to decompose ω_c in tetrahedra, the material polytope volume is

$$V_c^k = \sum_{tr \in \mathcal{T}_c} V(tr \cap \Pi_c^{k-}). \quad (9)$$

The position of the plane Π_c^k in ω_c is adjusted using a bisection method until $V(\omega_c^k) = V_c^k$, which fully defines Π_c^k .

3.2. Swept-face formulation

Consider a material variable ψ^k that must be remapped conservatively from a set of non-overlapping polygons $\{c\}$ to a new set of non-overlapping polygons $\{\tilde{c}\}$. The new value for ψ^k is defined as

$$\psi_{\tilde{c}}^k = \int_{\omega_c^k} \frac{\psi^k}{V^k} d\mathbf{x},$$

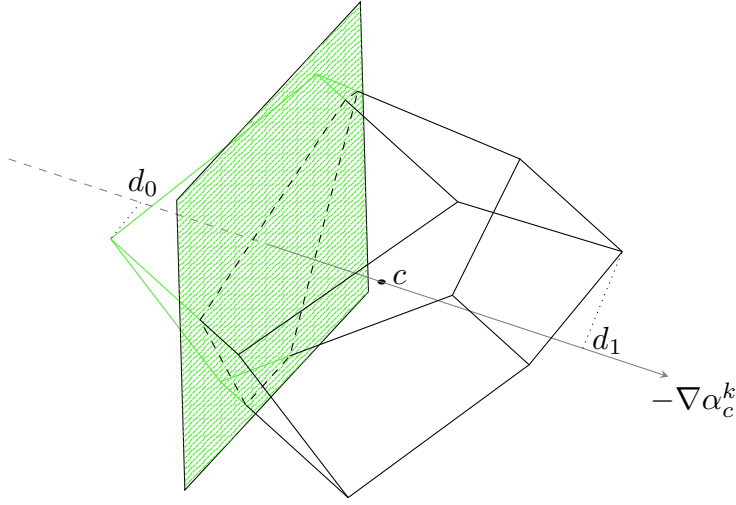


Figure 1: Schematic illustration of the computation of the oriented PLIC along the normal $\nabla \alpha_c^k$ passing through the cell centroid c and constrained between d_0 and d_1 . The green cell part represents the partial volume V^k occupied by material k in the half-space Π^{k-} .

where ω_c^k denotes the material polytope for k in $\tilde{c}$ and the term in the integral corresponds to the volumetric density of ψ^k . Without loss of generality, this formulation can be used for non-material-specific variables by dropping the k superscript here and in the following. Starting from the triangulation $\mathcal{T}(c)$ of the faces of the cell, we use the corresponding triangles in the regularized cell $\tilde{c}$ to form prisms denoted $\Omega_{\{p,p^+,p_f^*\}}$, where we recall that p and p^+ denote two consecutive nodes in the face triangulation and p_f^* denotes the face barycenter. For material k , we define the partial prism $\Omega_{\{p,p^+,p_f^*\}}^k = \omega_c^k \cap \Omega_{\{p,p^+,p_f^*\}}$, whose computation is explained later. The new variable ψ_c^k can then be computed thanks to a face sweeping strategy

$$\psi_c^k = \left(\int_{\omega_c^k} \frac{\psi_c^k}{V_c^k} d\mathbf{x} + \sum_{\mathcal{T}(c)} \int_{\Omega_{\{p,p^+,p_f^*\}}^k} \frac{\psi^k}{V^k} d\mathbf{x} \right),$$

where the last term in the right hand side corresponds to the contribution from the swept region. Applying this equation to the partial volume $\psi^k \equiv V^k$, which must be remapped conservatively for geometrical consistency, yields the new partial and total volumes

$$V_c^k = V_c^k + \sum_{\mathcal{T}(c)} V(\Omega_{\{p,p^+,p_f^*\}}^k), \quad V_c = \sum_k V_c^k, \quad \alpha_c^k = V_c^k / V_c,$$

where $V(\Omega_{\{p,p^+,p_f^*\}}^k)$ denotes the volume of $\Omega_{\{p,p^+,p_f^*\}}^k$. At this point, in the spirit of [4], an upwind strategy can be used to obtain

$$\int_{\Omega_{\{p,p^+,p_f^*\}}^k} \hat{\psi}^k d\mathbf{x} = \begin{cases} \int_{\Omega_{\{p,p^+,p_f^*\}}^k} \hat{\psi}_{c^+}^k(\mathbf{x}) d\mathbf{x} & \text{if } V(\Omega_{\{p,p^+,p_f^*\}}^k) > 0, \\ \int_{\Omega_{\{p,p^+,p_f^*\}}^k} \hat{\psi}_c^k(\mathbf{x}) d\mathbf{x} & \text{if } V(\Omega_{\{p,p^+,p_f^*\}}^k) < 0, \end{cases} \quad (10)$$

where we have defined $\hat{\psi}^k \equiv \psi^k / V^k$, c and c^+ denote the two neighboring Lagrangian cells that share the triangle $\{p, p^+, p_f^*\}$ and $\hat{\psi}_c^k(\mathbf{x})$, $\hat{\psi}_{c^+}^k(\mathbf{x})$ their respective reconstructions. In practice, piece-wise linear reconstructions of the variables on the Lagrangian grid are considered. It is a standard procedure and has

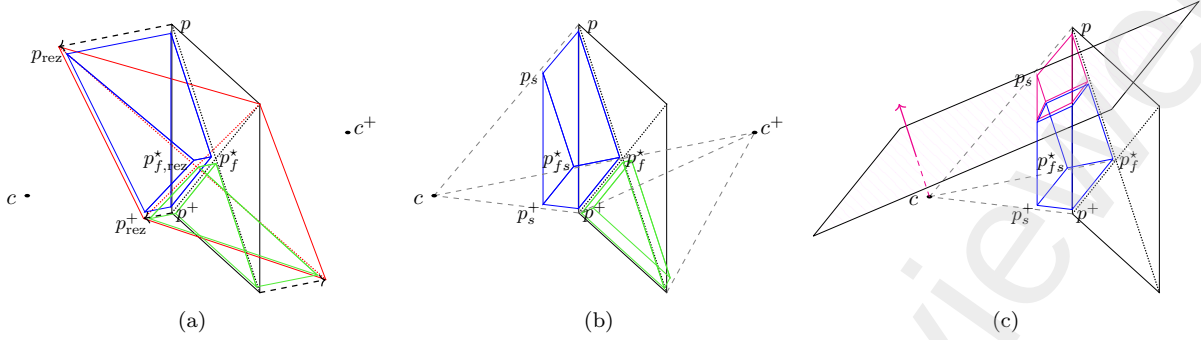


Figure 2: Decomposition of a swept face between cells c and c^+ in swept prisms for the ALE scheme. The Lagrangian nodes and edges are shown in black, and the rezoned ones in red (labeled with underscore *rez*). Two swept prisms are highlighted in blue and green. Panel (a) shows the original swept prisms, with Ω_{p,p^+,p_f^*} in blue. Panel (b) shows the equivalent surrogate swept-prisms, with $\bar{\Omega}_{p,p^+,p_f^*}$ representing Ω_{p,p^+,p_f^*} for the blue prism. This defines the new nodes $p_s, p_s^+, p_{fs}^+, p_{fs}^*$ for the surrogate prism. In this configuration, c is the donor cell for the blue prism, and c^+ is the donor cell for the green prism. In panel (c), the blue surrogate prism $\bar{\Omega}_{p,p^+,p_f^*}$ is sliced using the material plane Π_c^k to form the sub-polygon $\bar{\Omega}_{p,p^+,p_f^*}^k$. Slicing of the green surrogate prism would use $\Pi_{c^+}^k$.

been extensively used in [34, 29]. Here, it is written for partial quantities evaluated at their respective partial centroids

$$\hat{\psi}_c^k(\mathbf{x}) = \hat{\psi}_c^k + \left(\nabla \hat{\psi}_c^k \right)_c \cdot (\mathbf{x} - \mathbf{C}_c^k),$$

where $\hat{\psi}_c^k$ represents the known mean value in the cell, $\mathbf{C}_c^k$ is the centroid of polytope ω_c^k , and $\left(\nabla \hat{\psi}_c^k \right)_c$ a gradient of the quantity $\hat{\psi}^k$ evaluated in cell c . Following [29], these gradient terms can be computed using a least square procedure, here using a nodal stencil. The slope-limitation of these gradients is explained later.

3.3. Definition of swept material polytopes

We now define $\Omega_{\{p,p^+,p_f^*\}}^k$. First, we calculate the position and orientation of the planar interface Π^k . Second, we calculate the intersection of $\Omega_{\{p,p^+,p_f^*\}}$ with Π^k to define the polytope $\Omega_{\{p,p^+,p_f^*\}}^k$. These geometrical calculations can be difficult because $\Omega_{\{p,p^+,p_f^*\}}$ may not be convex. Following [35], the 3D calculation of material polytope properties is simplified by replacing the swept prism $\Omega_{\{p,p^+,p_f^*\}}$ by $\bar{\Omega}_{\{p,p^+,p_f^*\}}$, such that $V(\bar{\Omega}_{\{p,p^+,p_f^*\}}) = V(\Omega_{\{p,p^+,p_f^*\}})$ and $\bar{\Omega}_{\{p,p^+,p_f^*\}}$ is a convex prism formed by truncating the tetrahedron $\{p, p^+, p_f^*, c'\}$ with a plane parallel to $\{p, p^+, p_f^*\}$, where c' denotes the centroid of cell c' . The latter may be c or c^+ depending on the sign of $V(\Omega_{\{p,p^+,p_f^*\}})$. An example is shown for two prisms in Fig. 2 (a) transformed in two surrogate prisms in Fig. 2 (b). Defining these surrogate prisms requires one to find the points $\{p_s, p_s^+, p_{fs}^+, p_{fs}^*\}$ in the plane parallel to $\{p, p^+, p_f^*\}$ such that the polygon formed by $\{p, p^+, p_f^*, p_s, p_s^+, p_{fs}^+, p_{fs}^*\}$ has a volume of $V(\Omega_{\{p,p^+,p_f^*\}})$. For this purpose, one can note that $\mathbf{x}_{p_s} = (1-d)\mathbf{x}_p + d\mathbf{x}_{c'}$ (and the same relation for p^+ and p_f^*), where the normalized distance $d = 1 - \left(1 - V(\Omega_{\{p,p^+,p_f^*\}}) / V(\Omega_{\{p,p^+,p_f^*,c'\}}) \right)^3$, with $V(\Omega_{\{p,p^+,p_f^*,c'\}})$ the volume of the aforementioned tetrahedron $\{p, p^+, p_f^*, c'\}$. This relation can be obtained by writing the volume of the surrogate prism as that of the full tetrahedron minus that of the smaller tetrahedron $\{p_s, p_s^+, p_{fs}^+, c'\}$.

Once the surrogate prism is defined, we intersect $\Pi_{c'}^k$ with $\bar{\Omega}_{\{p,p^+,p_f^*\}}$ to form $\bar{\Omega}_{\{p,p^+,p_f^*\}}^k$ (see Fig. 2 (c)). We can then calculate the volume $V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k)$ and the centroid $\mathbf{C}(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k)$ using the methods described

in [36]. Finally, we can now express the integral in (10) as

$$\int_{\bar{\Omega}_{\{p,p^+,p_f^*\}}^k} \hat{\psi}_c^k(\mathbf{x}) d\mathbf{x} = V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) \left[\hat{\psi}_c^k + \left(\nabla \hat{\psi}_c^k \right)_c \cdot \left(C(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) - C_c^k \right) \right], \quad (11)$$

where c can be replaced by c^+ depending on the sign of $V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k)$. The term in the right brackets represents the interpolation of $\hat{\psi}_c^k$ at the centroid of the surrogate partial swept polytope $\bar{\Omega}_{\{p,p^+,p_f^*\}}^k$. Following [35], cell gradients $(\nabla \hat{\psi}_c^k)_c$ are slope-limited using a Barth-Jespersen procedure [37] evaluated at the nodes of c .

At this point we highlight more clearly the benefits of the surrogate prism approach. First, since by definition of the surrogate polytopes the centroid $C(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k)$ is inside the convex hull of cell c , the slope limitation guarantees that the interpolation term in (11) is bounded by the values of $\hat{\psi}^k$ on the nodal neighborhood of c . Second, the intersection calculations between the material interface $\Pi_{c'}^k$ and the surrogate prism are easier because the latter is by definition always a truncated prism with parallel triangular bases and flat quadrangular faces. Third, because the surrogate prisms are defined as being a subset of tetrahedra $\{p_s, p_s^+, p_f^*, c'\}$, which themselves tessellate the cell c' by definition, the surrogate prism approach is guaranteed to conserve partial volumes (to within the precision of the geometrical operations).

3.4. Conservative Remapping

The remapping algorithm is applied to a set of quantities ψ^k for which the remapping will be conservative by definition, since the swept-face algorithm ensures that what is gained by a cell is lost exactly by another. The remapping is applied to partial mass m^k , partial internal energy $\mathcal{E}^k = \varepsilon^k m^k$ and partial momentum $\mathbf{P}^k = m^k \mathbf{V}$. Note that for momentum, the projection is written in a local basis aligned with the flow direction as in Refs. [38, 39]. For simplicity, this is not expanded upon here as it does not change the formulation of the scheme. We separate the contributions to the remapped quantity ψ_c^k into a low-order contribution $\psi_c^{k,L}$ and a higher order contribution $\Delta\psi_c^k$. Following Eq. (11), the remapping reads

$$\begin{aligned} \psi_c^k &= \psi_c^{k,L} + \Delta\psi_c^k, \\ \psi_c^{k,L} &= \psi_c^k + \sum_{\mathcal{T}(c)} V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) \hat{\psi}_{c'}^k, \\ \Delta\psi_c^k &= \sum_{\mathcal{T}(c)} V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) \left(\nabla \hat{\psi}_c^k \right)_{c'} \cdot \left(C(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) - C_{c'}^k \right), \end{aligned} \quad (12)$$

where $\psi_c^{k,L}$ denotes the first-order projection and $\Delta\psi_c^k$ is the difference between the second and first-order projections. We recall that c' denotes either c or c^+ depending on the sign of $V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k)$, and that $\hat{\psi}_c^k = \psi_c^k / V_c^k$.

3.5. Convexity of the first order remapping: ALE CFL condition

Considering only the first-order case (setting higher-order contributions to zero in Eq. 12), the remapping of the per-volume contribution $\hat{\psi}_c^k$ can be written

$$\hat{\psi}_c^k = \hat{\psi}_c^k \left(\frac{V_c^k}{V_c^k} + \frac{1}{V_c^k} \sum_{\substack{\mathcal{T}(c) \\ \text{donor}}} V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) \right) + \frac{1}{V_c^k} \sum_{\substack{\mathcal{T}(c) \\ \text{receiver}}} V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) \hat{\psi}_{c^+}^k,$$

where donor prisms have $V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) < 0$ and receiver prisms have $V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) > 0$. We can rewrite this sum as

$$\hat{\psi}_c^k = \hat{\psi}_c^k \delta_1 + \sum_{\substack{tr \in \mathcal{T}(c) \\ \text{receiver}}} \delta_{tr} \hat{\psi}_{c^+}^k, \quad (13)$$

where the expression of coefficients δ_1 and δ_{tr} is straightforward. Eq. (13) is a convex combination under two conditions: $\delta_1 + \sum \delta_{tr} = 1$ and all $\delta > 0$. The first condition is satisfied by noting that $V_c^k + \sum_{\mathcal{T}(c)} V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) \equiv V_c^k$ by definition of the swept face algorithm and of the surrogate prisms. The second condition is satisfied for all δ_{tr} since by definition the receiver prisms have $V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) > 0$. The only condition that remains is then that $\delta_1 > 0$. Since $V_c^k > 0$, this is equivalent to requiring $V_c^k + \sum_{\substack{\mathcal{T}(c) \\ \text{donor}}} V(\bar{\Omega}_{\{p,p^+,p_f^*\}}^k) > 0$. This last condition can be seen as a CFL-like condition on the ALE rezoning, and states that the partial volume in the cell should not undershoot during the swept-face remapping due to a rezoning mesh that is too far from the Lagrangian mesh. We ensure that this condition is always satisfied by conducting sub-stepping of the ALE remapping. Under the ALE CFL-like condition, the first-order remapping in Eq. (12) is a convex combination for $\hat{\psi}_c^k$, which guarantees the condition

$$\min_{c' \in \mathcal{C}(\mathcal{P}(c))} \hat{\psi}_{c'}^k \leq \hat{\psi}_c^k \leq \max_{c' \in \mathcal{C}(\mathcal{P}(c))} \hat{\psi}_{c'}^k, \quad (14)$$

where the bounds are taken here on the nodal neighborhood $\mathcal{C}(\mathcal{P}(c))$ of the cell. Taking the example of partial mass, this condition ensures that the first-order partial density on the rezoned mesh (after remapping) cannot overshoot or undershoot its neighbor values evaluated on the Lagrangian mesh (before remapping). Unfortunately, that property is lost for the second-order scheme. This is because we cannot factor the donor contributions into a sum that is guaranteed to be positive. However, the convexity of the first-order algorithm is crucial for the following.

3.6. Flux Corrected Remapping

In order to recover convexity properties in the second-order remapping algorithm, a Flux-Corrected Remapping (FCR) procedure is applied, as was introduced for the 2D case in [39]. We will later show modifications to the FCR for total energy conservation, so that we first summarize the FCR procedure. The FCR procedure aims to bound the remapped quantities such that they respect condition (14). The total flux is modified as

$$\psi_c^k = \psi_c^{k,L} + d_c \Delta \psi_c^k, \quad (15)$$

where a limiting coefficient $d_c \in [0, 1]$ is introduced. The goal of the FCR procedure is to choose the maximum value of d_c such that the condition (14) is respected for volumetric quantities $\hat{\psi}_c^k$. Since we have proven that the first-order algorithm respects convexity, the procedure is guaranteed to have a solution at least for $d_c = 0$. In practice, the FCR is applied at least to $\hat{\psi}_c^k \equiv \rho_c^k, \rho_c^k \varepsilon_c^k$, and more conditions can be added if required. The reader can refer to [39] for the procedure used to derive the coefficients. We give an example for the partial internal energy (i.e. $\psi_c^k \equiv \mathcal{E}_c^k$, corresponding to $\hat{\psi}_c^k \equiv \rho_c^k \varepsilon_c^k$), which reads:

$$d_c \leq \frac{(\rho_c^k \varepsilon_c^k)_{\min} V_c^k - \mathcal{E}_c^{k,L}}{\Delta \mathcal{E}_c^k}, \quad (16)$$

where $(\rho_c^k \varepsilon_c^k)_{\min} \equiv \min_{c' \in \mathcal{C}(\mathcal{P}(c))} (\rho_{c'}^k \varepsilon_{c'}^k)$, and this condition corresponds to the lower bound and is only required when $\Delta \mathcal{E}_c^k < 0$.

3.7. Ensuring total energy conservation

So far we have explained how the remapping algorithm is applied to partial masses, partial internal energies, and momentum, which are then conserved. However, since we have defined the remapped cell velocities from a projection of momentum, it is not guaranteed that the kinetic energy is conserved. Since the total energy is updated from the kinetic and internal energy, this means that the current algorithm does not conserve total energy. This stems from the fact that we do not project total energy, but instead internal energy. A method to ensure the conservation of total energy in that framework was recently proposed in [40]. The method consists in projecting the kinetic energy in addition to the momentum, and correcting for the difference in the two by modifying the internal energy. In our multi-material framework, the corrected internal energy is then

$$\mathcal{E}_{\tilde{c},\text{corr}}^k = \mathcal{E}_{\tilde{c}}^k + \left(\mathcal{E}_{\tilde{c},\text{kin}}^k - \frac{1}{2} \frac{\mathbf{P}_{\tilde{c}}^k \cdot \mathbf{P}_{\tilde{c}}^k}{m_{\tilde{c}}^k} \right), \quad (17)$$

where $\mathcal{E}_{\tilde{c},\text{kin}}^k = (m^k \mathbf{V}^2/2)_{\tilde{c}}$ is the remapped partial kinetic energy. Ref. [40] shows that this correction ensures total energy conservation, and that the bounding of $\mathcal{E}_{\tilde{c},\text{corr}}^k$ is guaranteed at first-order. In order to conserve that property at second-order, we express an FCR condition on the internal energy taking into account the new definition $\mathcal{E}_{\tilde{c},\text{corr}}^k$. Expressing (17) using (15), condition (14) becomes

$$\begin{aligned} & m_{\tilde{c}}^{k,L} (\mathcal{E}_{\tilde{c}}^{k,L} + \mathcal{E}_{\tilde{c},\text{kin}}^{k,L}) - \mathbf{P}_{\tilde{c}}^{k,L} \cdot \mathbf{P}_{\tilde{c}}^{k,L} / 2 - (\rho_c^k \varepsilon_c^k)_{\min} m_{\tilde{c}}^{k,L} V_{\tilde{c}}^k + \\ & d_c \left(m_{\tilde{c}}^{k,L} (\Delta \mathcal{E}_{\tilde{c}}^k + \Delta \mathcal{E}_{\tilde{c},\text{kin}}^k) + \Delta m_{\tilde{c}}^k (\mathcal{E}_{\tilde{c}}^{k,L} + \mathcal{E}_{\tilde{c},\text{kin}}^{k,L}) - \mathbf{P}_{\tilde{c}}^{k,L} \cdot \Delta \mathbf{P}_{\tilde{c}}^k - (\rho_c^k \varepsilon_c^k)_{\min} \Delta m_{\tilde{c}}^k V_{\tilde{c}}^k \right) + \\ & d_c^2 (\Delta m_{\tilde{c}}^k (\Delta \mathcal{E}_{\tilde{c}}^k + \Delta \mathcal{E}_{\tilde{c},\text{kin}}^k) - \Delta \mathbf{P}_{\tilde{c}}^k \cdot \Delta \mathbf{P}_{\tilde{c}}^k / 2) \geq 0, \end{aligned} \quad (18)$$

and the corresponding condition on $\varepsilon_{\tilde{c},\text{max}}^k$ is obtained by substituting $\varepsilon_{\tilde{c},\text{min}}^k$ for $\varepsilon_{\tilde{c},\text{max}}^k$ and changing the sign of the inequality. Ensuring condition (19) is more complex than the usual FCR condition (see e.g. (16)). However, since (19) is a quadratic equation in d_c , it can be rewritten as $a_2 d_c^2 + a_1 d_c + a_0$ and the coefficients a_i can be analyzed to determine the appropriate minimum value in $[0, 1]$ that satisfies the condition. Notably, since we recover the first-order condition by setting $d_c = 0$, we have $a_0 \geq 0$ by definition.

4. An unstructured Lagrangian mesh as a forest of octrees

This section starts by introducing the structure of the Lagrangian mesh in the framework of adaptive mesh refinement features. The foundational idea is the following: the mesh is topologically represented by a collection of octants (i.e., logical cubes), of various AMR levels $l \in \mathbb{N}$ and nonconformally connected with each other through their faces. We impose that octants that are face, edge, or corner neighbors cannot differ by more than one AMR level, the so-called 2:1 corner-balance condition. Octants can be arbitrarily refined as long as the 2:1 corner-balance is respected. The root octants (that is, $l = 0$) are called *trees*. A regular (e.g., logically Cartesian) ensemble of cells is often called a *mesh block* in the meshing community. In that sense, a tree can always be thought of as a mesh block, since octants in the same tree are described by locally Cartesian topological coordinates. In practice, mesh blocks can be made of several trees as long as they are regularly arranged.

The resulting mesh topology is then composed of an ensemble of blocks (or trees), where the topology in each block can be thought of as semi-unstructured (since it is still logically cubic per tree, although with nonconformal faces and edges), and the most refined elements are octants (so-called *forest of octree*). The junction between octants at tree boundaries also satisfies the 2:1 corner-balance condition, but depending on the arrangement of the trees, the connectivity can become degenerate at these boundaries. This will be important for the ALE algorithms. Finally, the octants are partitioned between MPI ranks in a flexible manner and with only a few rules, as will be explained later on. The handling of the trees, octants, their partitioning, refinement, coarsening, and 2:1 balance operations are all performed with the *p4est* library [21].

So far, we have defined the octants as the base logical structure of the mesh topology construction. However, the equations resolved on our mesh through the Lagrangian framework are not formulated on the

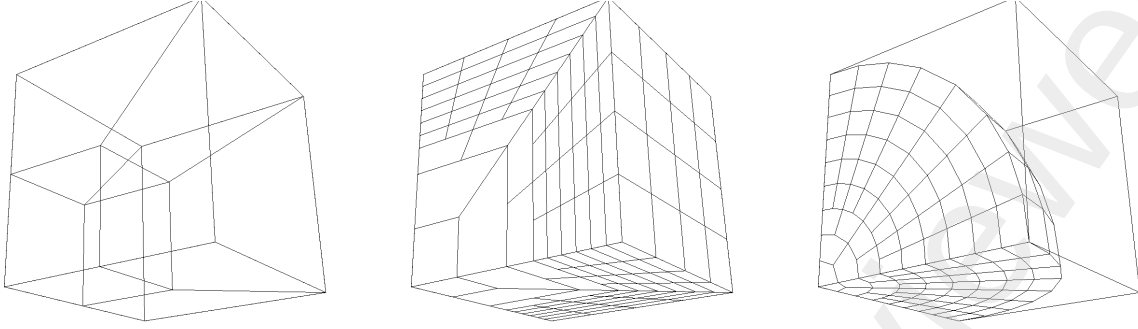


Figure 3: (left) example of a base connectivity structure made of 4 trees for a 8th sphere mesh shown in global logical coordinate space, (middle) refined root tree structure (for increased radial and angular resolution), (right) corresponding mesh in physical space.

collection of octants themselves. As introduced in Sec. 2, the flow is described by a discretization of the domain in polygons defined by their nodes. We must then transfer this forest of octree formalism to that of polygonal cells made of physical nodes. In addition, the hydrodynamics solver of Sec. 2 operates over the faces incident on the nodes. We must then construct a mesh of nodes and reconstruct the connectivity of faces incident on nodes, cells incident on nodes, and cells incident on faces.

The description of this framework is given in two parts. First, we describe the structure of the trees and their partition in Sec. 4.1, as this has implications for the other algorithms given in the paper. Then, we present the conversion from a forest of octrees to that of nodes and polygonal cells in Sec. 4.2.

4.1. Parallel mesh partitioning

The parallel mesh partitioning algorithm leverages the fact that, in *p4est*, constructing the connectivity information of each octree can be done using recursive algorithms applied on the tree structure. Consequently, octants can arbitrarily be partitioned among MPI ranks as long as each rank has a copy of the base tree structure. However, the most refined (*deepest*) octants in the forest are always stored in local processor memory.

The assignment of octrees to MPI ranks can be done in several ways. In hydrodynamics codes, cells are often partitioned to minimize the volume of parallel communications at boundaries between ranks. This often entails the use of graph-partitioning algorithms such as SCOTCH [41]. Since AMR modifies the mesh topology often, these algorithms would need to be used dynamically during the calculation, which is computationally expensive. As such, the partitioning in *p4est* relies on a simpler methodology that favors the ease of repartitioning the mesh during operation, at the cost of a larger communication layer. The partitioning relies on the logical indexing of trees and octants, which is introduced now.

4.1.1. Logical coordinates of trees and octants

The trees in the root structure are assigned indices $(\hat{i}, \hat{j}, \hat{k}) \in \mathbb{Z}$, which represent the position of the tree corners in the logical space. Octants are assigned local logical coordinates $(i, j, k) \in \mathbb{N}$, which represent the position of the corners of the octant in a local coordinate system of the tree they belong to. The combination of the local coordinates of the octants, refinement depth l , and global coordinates of their corresponding tree allows one to define global logical coordinates for the octants, denoted $(r_i, r_j, r_k) \in \mathbb{R}$. We give an example of a spherical mesh topology represented using their global logical coordinates in Fig. 3.

The set of all octants $\mathcal{O}$ with $\text{card}(\mathcal{O})$ elements is numbered globally using their logical indices. To that end, trees are lexicographically numbered such that $\hat{i} < \hat{j} < \hat{k}$. Within a tree, octants are also lexicographically numbered such that $i < j < k$. Combining the two, octants are then numbered first by the tree they belong to and then by their coordinates in the tree. Note that because two adjacent trees may not be oriented in the same way, this ordering does not yield $r_i < r_j < r_k$ in general. Now that all octants have been numbered, we can describe the partitioning algorithm.

4.1.2. Partitioning of octants

Assuming $\mathcal{N}_{\text{tot}}$ MPI ranks, the octants are assigned a contiguous section of the list of all octants such that each MPI rank *owns* approximately $\text{card}(\mathcal{O})/\mathcal{N}_{\text{tot}}$ octants (without weighting). Since the list of all octants is contiguously divided, there is a straightforward mapping between local cell numbering and global cell numbering. For example, if we denote by O_i the total number of octants owned by rank i , and $O_{[1:i]} = \sum_{j \in [1,i]} O_j$, then the octants for rank $i + 1$ have global numbers in the range $[O_{[1:i]} + 1, O_{[1:i+1]}]$. This straightforward mapping significantly simplifies mesh repartitioning (dynamically changing the partition of the mesh cells on the MPI ranks during AMR operation), mesh fusion (joining different meshes together for producing single-output files or restarting a calculation on a different number of MPI ranks), heterogeneous decomposition (using a different MPI decomposition for ranks solving the hydrodynamics and ranks solving other physics, and the construction of the cell ghost layer).

During the calculation, when the number of mesh cells changes due to coarsening and refinement, the mesh can be re-balanced by following the partitioning principle given above. While the computation of the new partition of mesh cells with MPI ranks is trivial, we add the condition that a group of 8 refined octants that originate from the same parent be always present on the same MPI rank. That functionality is provided by *p4est* and simplifies the coarsening algorithm. However, it requires to conduct a partitioning operation after each regridding operation, otherwise the condition could be lost during successive coarsening steps. After the new partition is computed, the kinematic and thermodynamic values associated with the mesh cells must be shipped to their new MPI rank owners, and eventually parts of the code must be re-initialized. An example of mesh partitioning and dynamic repartition is given in section 6.5 for the triple-point test case.

4.2. Converting octants to unstructured cells

In this section, we describe how the parallel tree of octants is converted to a domain-decomposed mesh of polygonal cells.

4.2.1. Distinction between octants and mesh cells

Some definitions are now presented. These terms are used extensively in the next sections.

- The term *octant* employed up to now always refers to the topological hexahedron used in building the mesh topology.
- The term *corners* always refers to the 8 logical nodes that describe the octant. As shown later, they also correspond to the first 8 nodes of the mesh cells in their local numbering.
- A *hanging node* is a node that touches the face or edge of an octant but that is not one of the corners of that octant.
- A *node* and *cell* refers to the polygonal structures of the mesh, not the tree of octants.

Following these definitions, we note that since an octant is only described by 8 corners, a hanging node is never part of the octant it is hanging in; instead, it is a corner node in another octant. This difference is a fundamental one and must be highlighted: all nodes do not have physical meaning for all octants in the forest of octree framework, whereas all nodes have geometrical meaning for all cells in the Lagrangian framework. As such, *p4est* does not have the required functionality pertaining to nodes. The task is then to *convert* the 8-corners octants to unstructured cells, as illustrated in Fig. 4. Another difficulty is introduced by the parallel framework: since hanging nodes cannot be owned by the octants in which they hang (since they are not a corner in it), hanging nodes on MPI boundaries are not present in memory of MPI ranks in which the node is hanging.

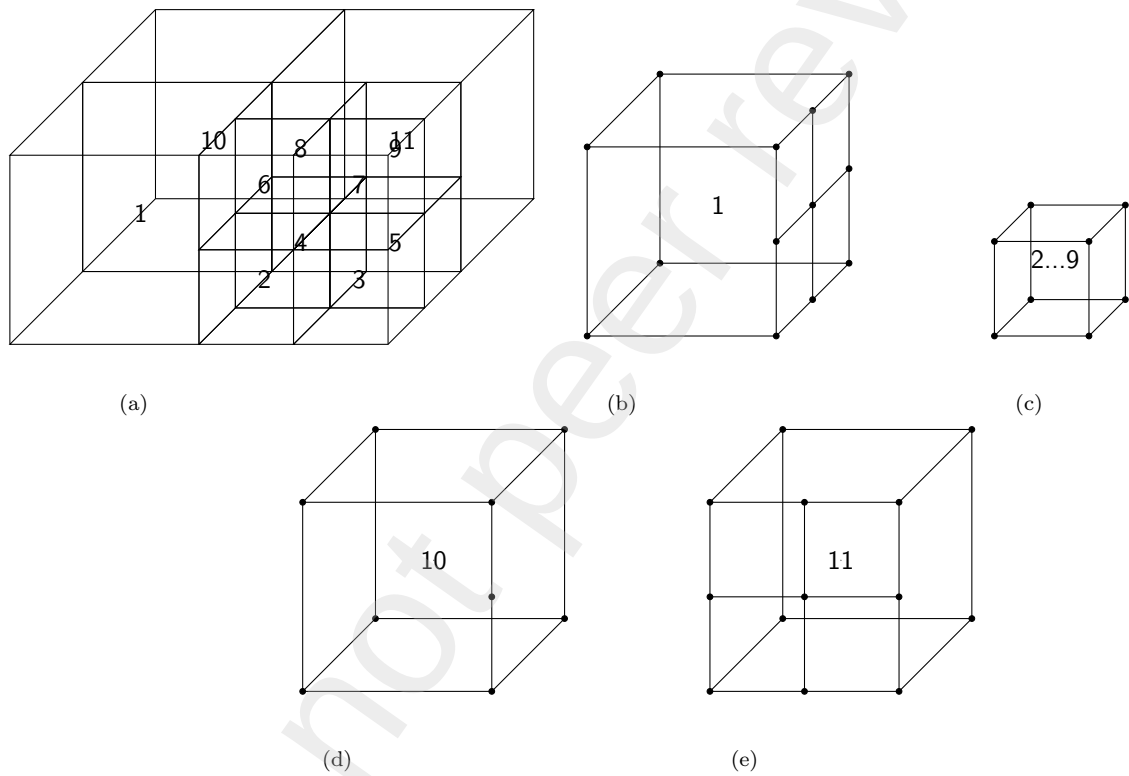


Figure 4: Schematic illustration of the correspondence between (a) a p_4est forest of octrees with 4 root trees with a single tree refined by one level, and (b-e) the corresponding polygonal cells and their faces and nodes for the hydrodynamics. The resulting cells are no longer hexahedra, e.g. converted cells 1 and 11 have 9 faces, 5 of which have 4 nodes, and 4 of which have 5 nodes.

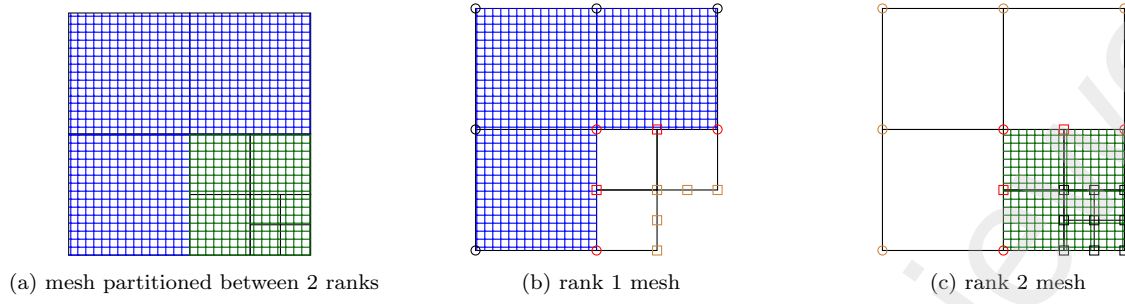


Figure 5: Example of a partition of mesh (a) between rank 1 cells in blue and rank 2 cells in green. The actual mesh constructed for each rank is shown in panels (b) and (c). In these two panels, nodes are shown as symbols. Circle denote nodes owned by rank 1 and squares are nodes owned by rank 2. Blacks symbols are non-duplicated internal nodes, red symbols are duplicated internal nodes and orange symbols are external nodes. Note that for illustrative purposes, the partition here is not balanced in terms of load.

4.2.2. Definition of the remote mesh structure

The computation of many physical quantities requires one to gather information from remote MPI ranks. Without entering into all the details, we introduce some of the concepts required in the next sections, notably for handling hanging nodes on MPI boundaries. The second-order algorithms of Sec. 2 require to access quantities in cells that share a node. As such, the mesh in each local rank is expanded to contain one layer of cells that share a node with cells in the owned partition. This additional layers adds news cells and nodes, which are grouped in the following categories also described in Fig. 5:

- a cell is *internal* if it is owned by the current MPI rank, it is otherwise *external*
- a node is internal if it is incident on an internal cell, it is otherwise external (note that this is different from the *p4est* definition)
- an internal node may be termed *duplicated* if it is internal in two different ranks
- a non-duplicated internal node is always owned by the current MPI rank
- a duplicated internal node belongs to the lowest MPI rank of the cells incident on it, only if that node is not hanging in that cell

Most operations can be carried out on internal nodes regardless of their duplication status, as long as the external nodes are synchronized. However, some AMR operations introduced later require the distinction between owned and non-owned duplicated internal nodes.

4.2.3. Polygonal mesh construction

From the *p4est* interface we obtain the following:

- position in logical space of the owned octants,
- list of octant neighbors accross faces, edges and node, only between internal octants or from internal to external octants.

The first step is to reconstruct octant neighborhood information through faces, edges and nodes for octants in the external layer, both between external octants and from external to internal octants. This step is required for the node numbering algorithm, which must also operate in the external layer in order to ascribe physical meaning to faces shared by external cells. The missing connectivity information is reconstructed using the global octant numbering of Sec. 4.1.2. The search is partially speeded up by using the existing information relating internal to external octants.

We now define an *augmented* polygonal cell template made of 26 nodes and with a local numbering scheme illustrated in Fig. 6. The augmented cell is an intermediate build step and will be simplified into

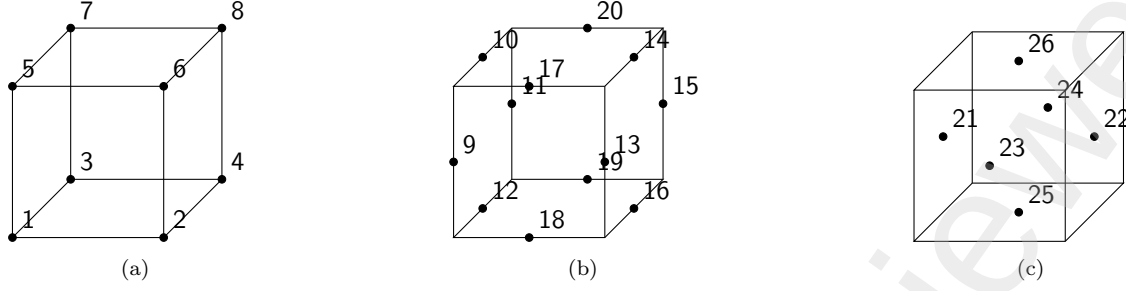


Figure 6: Cell-local numbering scheme used exclusively for building the polygonal cells from the *p4est* octants. Numbering of (a) the corner nodes, following the *p4est* lexicographic ordering, (b) the edge hanging nodes, and (c) the face hanging nodes. The face hanging nodes are numbered such that subtracting 20 gives the local face number in the *p4est* convention.

a normal cell by removing non-numbered local nodes later. The most important feature is that the 8 first local node numbers of the cell correspond exactly to the *p4est* local lexicographic ordering of the octant corner, which will be important for the AMR and ALE algorithms. Since a polygonal cell in the mesh has at least as many degrees of freedom as an octant (i.e. is at least an hexahedra), these 8 corner nodes are always defined in the augmented cell.

We number the nodes from the full octant connectivity in three passes. In the first pass, we loop over all (internal + external) octants and number the 8 corner nodes of each augmented cell, only if these nodes belong to an internal cell. Note that in this approach, we will number the hanging nodes on the internal cells which are corner nodes of external cells (e.g. blue node in Fig. 7). Proceeding this way allows one to define all internal nodes in the first pass, such that we have a contiguous chunk of nodes in memory that are all internal. The rest of the nodes to be labelled are external nodes. In the second pass, we loop over external cell and number every corner node that has not been labeled before. We also use the connectivity to find nodes in those cells that share at least an edge or a face with another external cell (these are hanging nodes by definition). At that point, we synchronize the augmented cells over the domain decomposition to prepare for the third pass. Finally, we conduct the third pass in which we find nodes in the external cells that are hanging between external nodes. An example is given in Fig. 7 as a green node, and in Fig. 5 (b) as some of the orange square nodes.

Once the nodes are labeled, the augmented cells are simplified into polygonal cells by removing un-numbered nodes. We then build the remaining of the connectivity information required; cells sharing a node, cells sharing a face, global face numbering and list of nodes per face, etc. At the end of the conversion process, a polygonal cell is defined by 6 to 24 faces. Depending on the configuration, the faces may be composed from 4 to 8 nodes.

This approach where we do not use the *p4est* memory structures but duplicate our own to construct a physical mesh of polygons does introduce some overhead. Indeed, the connectivity information is duplicated and each time the topology of the mesh changes (through repartition or regridding), we must deallocate our arrays and rebuild the mesh and the physics variables arrays. On the contrary, *p4est* uses flexible linked-list-like structures in which elements can be inserted and removed on the fly. While the *p4est* approach does not require these re-allocations, manipulating data and computing array operations on linked-list-like structures is much less efficient than conventional multi-dimensional Fortran arrays, since the latter provide contiguous memory access for many relevant operations. Linked-lists also add their own overhead with the definition of next/previous elements and their handling. As such, we estimate that our added allocations/deallocations/copies are compensated by the easier access to mesh and physics data in the rest of the computational loop. In addition, the repartition and regridding steps are typically strided, e.g. at least every 10 hydrodynamics steps, such that costs incurred by the conversion process are largely reduced.

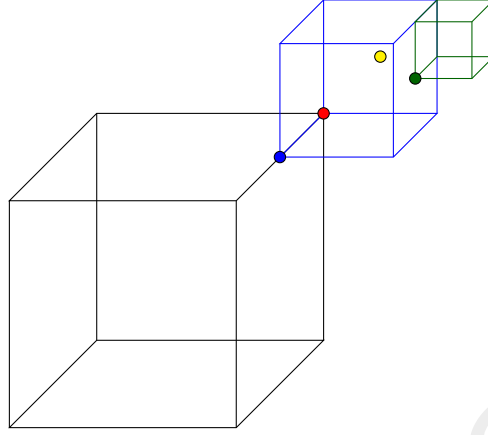


Figure 7: Illustration of difficulties that may arise from converting the parallel forest of octants formalism of *p4est* to a domain-decomposed mesh of polygonal cells defined by their nodes. The black and blue octants belong to MPI ranks denoted 1 and 2, respectively. The blue node is an edge hanging node in the black cell, but is not present in the *p4est* memory of rank 1 because it is owned by the blue octant in which it is a corner. The red node is a corner node both for the blue and black cell. However, it is not present in memory for both ranks 1 and 2. The yellow node cannot exist since it would mean that the forest is not 2:1 corner balanced. The green node is face-hanging in the blue cell. Depending on the configuration, it may not be owned by rank 2 either (e.g. if a rank 3 owns the green cell right of the blue cell), such that the green node may not be in memory of rank 2.

5. Algorithms for AMR of Lagrangian meshes

In the previous section, we have described the structure and construction of a domain-decomposed unstructured polygonal mesh from a parallel forest of octrees. In this section, we now describe the algorithms implemented for AMR operations performed on this type of Lagrangian meshes. First, some terms need to be defined. The operation of refining and coarsening a mesh is called *regridding*. The mesh that is being re-gridded is called *outgoing* mesh and denoted $\mathcal{M}_{\text{out}}$, whereas the new re-gridded mesh is called *ingoing* mesh and denoted $\mathcal{M}_{\text{in}}$. The definition of new node positions in the ingoing mesh is given in Sec. 5.1. We then detail the remapping of quantities from the outgoing to the ingoing mesh in Sec. 5.2. The criteria used to re-grid the mesh are given in Sec. 5.3. Finally, a summary of the Lagrangian AMR algorithm is given in Sec. 5.4.

5.1. Definition of re-gridded nodes

The algorithms used to define node positions during regridding are now presented. During refinement, new nodes may be added to the mesh. During coarsening, the nodes may be removed. The addition and removal of nodes redefines the geometry of the underlying cells. For the AMR step to be conservative in terms of mass and energy, it is key that the regridding does not change the volume of the mesh. This statement is mostly trivial in the Eulerian framework, but it is not for 3D Lagrangian cells, for two reasons. First, faces in 3D are not flat in general, such that we rely on triangulations to compute volumes. However, the topology of these triangulations will change during the refinement process, which can change the local volume. Second, removing nodes during coarsening removes degrees of freedom in the cell geometry, which may also change the local volume. We start by giving some sufficient conditions for volume conservation for 3D Lagrangian AMR in Secs. 5.1.1 and 5.1.2, alongside algorithms for computing the re-gridded mesh nodes.

5.1.1. Volume-conservative refinement for 3D Lagrangian cells

We first consider the case of cell refinement. The refinement of an outgoing cell into 8 ingoing cells does not always add new nodes, because adjacent cells may already be refined to the same AMR level and because of the 2:1 corner-balance condition. We denote the outgoing cell as $c \in \mathcal{M}_{\text{out}}$, and the eight ingoing

refined cells are denoted as d and numbered in the list $\mathcal{R}_c \in \mathcal{M}_{\text{in}}$. We also define $\mathcal{A}_d$ as the list of cells in $\mathcal{M}_{\text{out}}$ that are an ancestor of $d \in \mathcal{M}_{\text{in}}$. The refinement adds, at most, 6 nodes defined from the faces of c , 12 nodes defined from the edges of c , and a node defined from all nodes of c . For refinement to preserve total volume, we must have $V_c = \sum_{d \in \mathcal{R}_c} V_d$. This condition can be met in different ways, depending on how the new nodes are defined. It is reasonable here to add the following constraints: nodes created from the outgoing edges should be on the edge themselves, and the central node should be in the convex hull of the outgoing cell. Note that these constraints would not be met for higher-order shape elements such as spherical hexahedra. The only remaining degree of freedom is the definition of the nodes introduced from the faces of c and denoted $\hat{p}_f$.

In the 3D EUCCLHYD framework, the numerical scheme and computation of V_c relies on the triangulation of the faces of c through the introduction of a virtual node at the faces' barycenters, p_f^* (the star symbol denotes that this is not a physical node; see also Sect. 2). We recall that faces in our AMR framework may have 4 to 8 nodes — 4 nodes for a face with no refined edges, 8 nodes for a nonrefined face with 4 refined edges. The contribution of face f to the volume of a cell incident on it, denoted V_f , is

$$6V_f = \sum_{p \in \mathcal{P}(f)} \mathbf{x}_p \cdot (\mathbf{x}_{p^+} \times \mathbf{x}_{p_f^*}), \quad \mathbf{x}_{p_f^*} = \frac{1}{\text{card}(\mathcal{P}(f))} \sum_{p \in \mathcal{P}(f)} \mathbf{x}_p, \quad (19)$$

where p^+ denotes the next node in the oriented triangulation of the face. An example of face triangulation for a face with pre-existing edge-hanging nodes is shown in Fig. 8 (a).

After refinement, there are 4 ingoing cells with a face defined from the outgoing face f . By virtue of the 2:1 corner-balance, these new faces necessarily have 4 nodes. For each of these faces, one node is one of the four octant corners of the cell of the outgoing face, and is denoted p_o^g (with $g \in [1, 4]$). Two other nodes were created on the segments $[p_o^{g-1}; p_o^g]$ and $[p_o^g; p_o^{g+1}]$, we denote these p_o^{g-} and p_o^{g+} (note that $p_o^{g-} \equiv p_o^{g-1+}$ and $p_o^{g+} \equiv p_o^{g+1-}$). The final node is shared by the 4 new faces, and is the degree of freedom introduced earlier, $\hat{p}_f$. The definition of these nodes is indicated in Fig. 9 and can also be found in Fig. 8(e). Defining $\hat{V}_f$ as the volume contribution to these 4 ingoing faces created from f during refinement, one writes

$$6\hat{V}_f = \sum_{g \in [1, 4]} \left(\mathbf{x}_{p_o^g} \cdot (\mathbf{x}_{p_o^{g+}} \times \mathbf{x}_{p_o^{g*}}) + \mathbf{x}_{p_o^{g+}} \cdot (\mathbf{x}_{\hat{p}_f} \times \mathbf{x}_{p_o^{g*}}) + \mathbf{x}_{\hat{p}_f} \cdot (\mathbf{x}_{p_o^{g-}} \times \mathbf{x}_{p_o^{g*}}) + \mathbf{x}_{p_o^{g-}} \cdot (\mathbf{x}_{p_o^g} \times \mathbf{x}_{p_o^{g*}}) \right),$$

$$\mathbf{x}_{p_o^{g*}} = \frac{1}{4} \left(\mathbf{x}_{p_o^g} + \mathbf{x}_{p_o^{g+}} + \mathbf{x}_{\hat{p}_f} + \mathbf{x}_{p_o^{g-}} \right),$$

which can be simplified by using the vector identities $a \cdot (b \times a) = 0$ and $a \cdot (b \times b) = 0$ as

$$12\hat{V}_f = \sum_{g \in [1, 4]} \left(\mathbf{x}_{p_o^g} \cdot (\mathbf{x}_{p_o^{g+}} \times \mathbf{x}_{\hat{p}_f}) + \mathbf{x}_{\hat{p}_f} \cdot (\mathbf{x}_{p_o^g} \times \mathbf{x}_{p_o^{g-}}) + \mathbf{x}_{p_o^{g+}} \cdot (\mathbf{x}_{p_o^{g-}} \times \mathbf{x}_{\hat{p}_f}) + \mathbf{x}_{p_o^{g-}} \cdot (\mathbf{x}_{p_o^g} \times \mathbf{x}_{p_o^{g+}}) \right). \quad (20)$$

For the refinement step to conserve volume, we must have $V_f = \hat{V}_f$, for any number of nodes in the outgoing face $\mathcal{P}(f)$. Once again, there is no unique method to achieve this condition. Without adjustments, i.e. using the preexisting edge-hanging nodes, this condition will not be respected. An example is given in Fig. 8 (b). Here we propose one solution that is compatible with our line-sweep ALE algorithm described in Part II of this study. The method is as follows:

1. the newly defined edge nodes are set to the barycenters of the octant corner nodes they depend on, i.e. $\mathbf{x}_{p_o^{g+}} = (\mathbf{x}_{p_o^g} + \mathbf{x}_{p_o^{g+1}})/2$ and similarly for p_o^{g-} .
2. the degree of freedom $\hat{p}_f$ is set at the barycenter of the four octant corner nodes of the outgoing face, i.e. $\mathbf{x}_{\hat{p}_f} = (1/4) \sum_{g \in [1, 4]} \mathbf{x}_{p_o^g}$
3. the already existing non-corner nodes in $\mathcal{P}(f)$ (i.e. edge-hanging nodes in the edges of f) are moved to the barycenters of the octant corner nodes they depend on prior to refinement

Items 1 and 2 are not necessary for volume conservation, but produce more regular refined elements. Item 3 is required for volume conservation, although a less strict and sufficient condition would be that edge-hanging

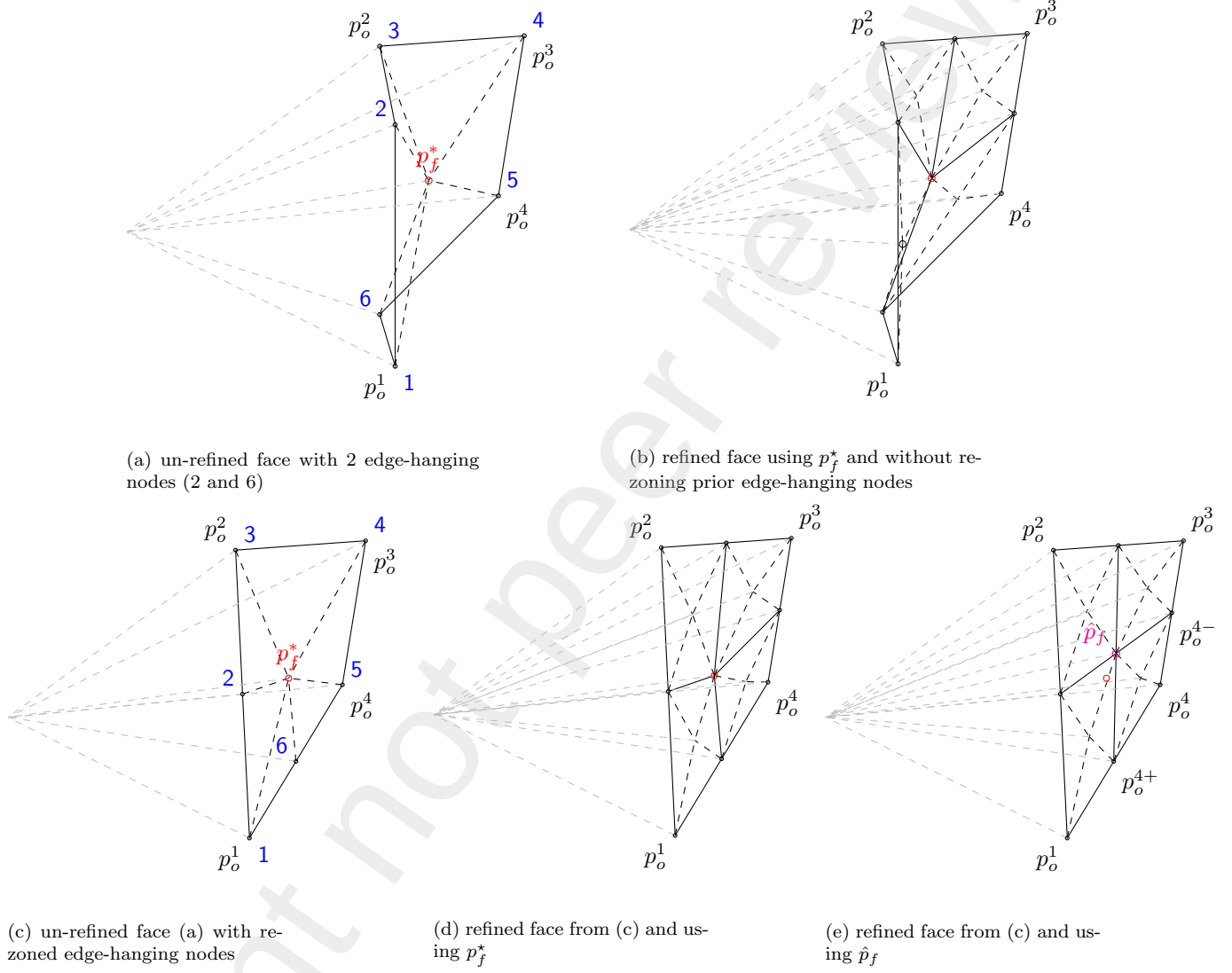


Figure 8: Illustration of the issue of volume conservation arising for face refinement. A non-refined face with pre-existing edge-hanging nodes (a) can be refined using these hanging nodes and the face barycenter p_f^* , giving (b). In the general case, the volume incident on (a) and (b) using the triangulation of Sec. 2 is different. Face (a) can be preprocessed to rezone edge-hanging nodes at their barycenter, as shown in (c). The face can then be refined using (d) p_f^* or (e) $\hat{p}_f$, yielding the same volume incident on the face. In (a) and (c), the blue numbers indicate the face nodes. In all panels, the notation p_o^i with $i \in [1, 4]$ denotes the octant face nodes. The red node indicates the barycenter of the un-refined face, and the magenta node indicates the barycenter of the un-refined octant face nodes.

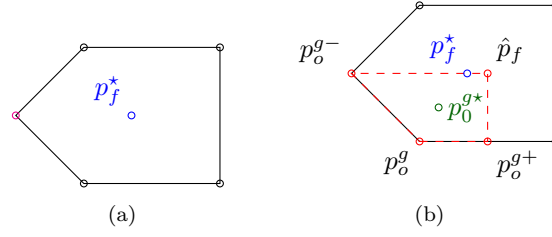


Figure 9: Labelling of the nodes during refinement of a face. (a) un-refined face made of 5 nodes. In that example, the magenta node is already edge-hanging, e.g. the cell is adjacent on that edge with a refined cell. The blue node p_f^* is the virtual face barycenter as defined for the EUCCLHYD scheme. (b) node labelling around p_0^g after refinement of the face. p_0^g is one of the 4 corner nodes. p_0^{g+} and p_0^{g-} are edge-hanging nodes incident on p_0^g in that cell, noting that p_0^{g-} was a pre-existing node. The red nodes define one of the 4 new faces. p_f is the degree of freedom introduced during the refinement, and is not necessarily at the same position as the old face barycenter p_f^* .

nodes be rezoned anywhere onto their parent segment. Item 3 is usually respected for Eulerian meshes at initialization. Since it involves rezoning the mesh node positions, it is executed as preprocessing through an ALE step in cells undergoing refinement. An example is given in Fig. 8 (c). A similar preprocessing step will be necessary for coarsening. Item 3 allows to rewrite the volume incident on the un-refined face V_f as

$$6V_f = \sum_{g \in [1,4]} \mathbf{x}_{p_0^g} \cdot (\mathbf{x}_{p_0^{g+1}} \times \mathbf{x}_{p_f^*}), \quad (21)$$

where the definition of $\mathbf{x}_{p_f^*}$ is unchanged and still obtained from $\text{card}(\mathcal{P}(f))$ nodes. Inserting in Eq. (20) the expressions for $\mathbf{x}_{p_0^{g+}}$ and $\mathbf{x}_{p_f}$ from items 1 and 2, we obtain the following equation for $\hat{V}_f$

$$6\hat{V}_f = \frac{1}{2} \sum_{g \in [1,4]} \mathbf{x}_{p_0^g} \cdot (\mathbf{x}_{p_0^{g+1}} \times \mathbf{x}_{p_0^{g-1}}). \quad (22)$$

Assuming that f has 4 nodes, then we can write $\mathbf{x}_{p_f^*} = (1/4) \sum \mathbf{x}_{p_0^g} = \mathbf{x}_{p_f}$ and by inserting this expression in Eq. (21) for V_f it is straightforward to show that we get $V_f = \hat{V}_f$. Assuming that f has more than 4 nodes, then $\mathbf{x}_{p_f^*} \neq \mathbf{x}_{p_f}$. We show the calculation in case of a face with 5 nodes and it is straightforward to generalize to the other cases. From item 3 above we have that the 5-th node is at the barycenter of an edge. Without loss of generality, we assume that node is in on the first edge, i.e. $\mathbf{x}_{p_0^{1+}} = \mathbf{x}_{p_0^{2-}} = (\mathbf{x}_{p_0^1} + \mathbf{x}_{p_0^2})/2$. Inserting the corresponding expression for $\mathbf{x}_{p_f^*}$ in Eq. (21) for V_f one obtains

$$\begin{aligned} 6V_f &= \sum_{g \in [1,4]} \mathbf{x}_{p_0^g} \cdot (\mathbf{x}_{p_0^{g+1}} \times \mathbf{x}_{p_f^*}), \quad \mathbf{x}_{p_f^*} = \frac{1}{5} \left(\mathbf{x}_{p_0^1} + \mathbf{x}_{p_0^{1+}} + \mathbf{x}_{p_0^2} + \mathbf{x}_{p_0^3} + \mathbf{x}_{p_0^4} \right), \\ &= \frac{1}{5} \sum_{g \in [1,4]} \mathbf{x}_{p_0^g} \cdot \left(\mathbf{x}_{p_0^{g+1}} \times \left(\frac{3}{2} \mathbf{x}_{p_0^1} + \frac{3}{2} \mathbf{x}_{p_0^2} + \mathbf{x}_{p_0^3} + \mathbf{x}_{p_0^4} \right) \right), \\ &= \frac{1}{5} \left(\mathbf{x}_{p_0^1} \cdot (\mathbf{x}_{p_0^2} \times \mathbf{x}_{p_0^3}) + \mathbf{x}_{p_0^1} \cdot (\mathbf{x}_{p_0^2} \times \mathbf{x}_{p_0^4}) + \mathbf{x}_{p_0^2} \cdot (\mathbf{x}_{p_0^3} \times \mathbf{x}_{p_0^4}) + \mathbf{x}_{p_0^2} \cdot (\mathbf{x}_{p_0^3} \times \frac{3}{2} \mathbf{x}_{p_0^1}) + \right. \\ &\quad \left. \mathbf{x}_{p_0^3} \cdot (\mathbf{x}_{p_0^4} \times \frac{3}{2} \mathbf{x}_{p_0^1}) + \mathbf{x}_{p_0^3} \cdot (\mathbf{x}_{p_0^4} \times \frac{3}{2} \mathbf{x}_{p_0^2}) + \mathbf{x}_{p_0^4} \cdot (\mathbf{x}_{p_0^1} \times \frac{3}{2} \mathbf{x}_{p_0^2}) + \mathbf{x}_{p_0^4} \cdot (\mathbf{x}_{p_0^1} \times \mathbf{x}_{p_0^3}) \right), \\ &= \frac{1}{5} \left(\frac{5}{2} \sum_{g \in [1,4]} \mathbf{x}_{p_0^g} \cdot (\mathbf{x}_{p_0^{g+1}} \times \mathbf{x}_{p_0^{g-1}}) \right) = 6\hat{V}_f. \end{aligned} \quad (23)$$

which ends the demonstration that the face splitting algorithm preserves volume for refinement of the 3D Lagrangian cells.

As hinted before, the added nodes may correspond to pre-existing nodes in the outgoing mesh; nodes that were already face-hanging or edge-hanging in the un-refined cell faces. When defining the re-gridded node coordinates using the new numbering on the new mesh topology, the coordinates of pre-existing nodes must be recovered. A two-pass algorithm (Alg. 1) is used to define the ingoing node coordinates. The first pass defines the known corner nodes of the ingoing cells on the internal mesh. The second pass processes the added nodes on the internal mesh, handling detection of pre-existing nodes and definition of new coordinates. Definition of the nodes in the ghost layer requires two MPI synchronizations; first, synchronization of nodes hanging on faces/edges between an internal and external cell, and second, synchronization of nodes on faces that are not incident on internal cells.

Algorithm 1 Algorithm for computing coordinates of ingoing nodes.

```

for  $d \in \mathcal{C}_{\text{int}}^{\text{in}}$  do
  if  $d$  is a refined cell then
    Copy the corner node of local number  $i$  from the outgoing cell  $c$  to ingoing cell  $d$ , where  $i$  is the
    index of  $d$  in  $\mathcal{R}_c$  (see Fig. 10 (a)). This defines one corner node in  $d$ . The other 7 may be either new
    nodes or pre-existing nodes in the outgoing mesh.
  else if  $c$  is a coarsened cell then
    Loop over the 8 outgoing cells and copy the 8 corresponding shared corner nodes (as above, see
    Fig. 10 (a)). This fully defines the corner nodes of  $d$ .
  else
    Copy the 8 corner nodes from the outgoing cell. Cell  $d$  may still have additional nodes to be
    defined, which are edge or face hanging.
  end if
end for
for  $d \in \mathcal{C}_{\text{int}}^{\text{in}}$  do
  if  $d$  is a refined cell then
    Loop over the 3 faces sharing the corner node  $i$ , where  $i$  is the index of  $d$  in  $\mathcal{R}_c$ . If the node
    diagonally opposed to  $i$  in that face has not been processed (i.e. it is not a corner node for any other
    internal cell), define its coordinates  $\mathbf{r}_f$  from the four corner nodes in the outgoing mesh that constituted
    the corresponding outgoing face (see Fig. 10 (b)).
    Loop over the 3 edges sharing the corner node  $i$ , where  $i$  is the index of  $d$  in  $\mathcal{R}_c$ . If the second node
    in the edge has not been processed, define its coordinates  $\mathbf{r}_e$  from the two corner nodes in the outgoing
    mesh that constituted the corresponding outgoing edge (see Fig. 10 (c)).
    Define the node diagonally opposite to  $i$  in  $d$  as the barycenter  $\mathbf{r}_b$  of the 8 corner nodes in the
    outgoing cell  $c$  (see Fig. 10 (d)).
  end if
end for

```

5.1.2. Volume-conservative coarsening for 3D Lagrangian cells

We now consider the case of cell coarsening. In the Lagrangian or ALE case, eight outgoing cells flagged for coarsening are, in the general case, not a tessellation of the ingoing coarsened cell. This is because the coarsened cell has fewer nodes, such that its geometry may not match the outgoing geometry. An example is given in Fig. 11. This tessellation discrepancy implies that the remapping step from Fig. 11 (a) to (b) must be adapted. In theory, an *exact* projection algorithm, i.e., that does not approximate the geometry (contrary to Sec. 3.1) can handle this situation. However, there is significant complexity because the topology of the mesh before and after the coarsening has changed. Instead, we propose a simple solution that consists in inserting an ALE step during the AMR remapping, similarly to the refinement case.

After cells are flagged for coarsening, and after the 2:1 balance step is performed, we perform an ALE remapping step using the swept-face method of Sec. 3.1. The nodes positions for this intermediate remapping step are defined such that they reproduce the conditions in which the cells were refined, as defined in Sec.

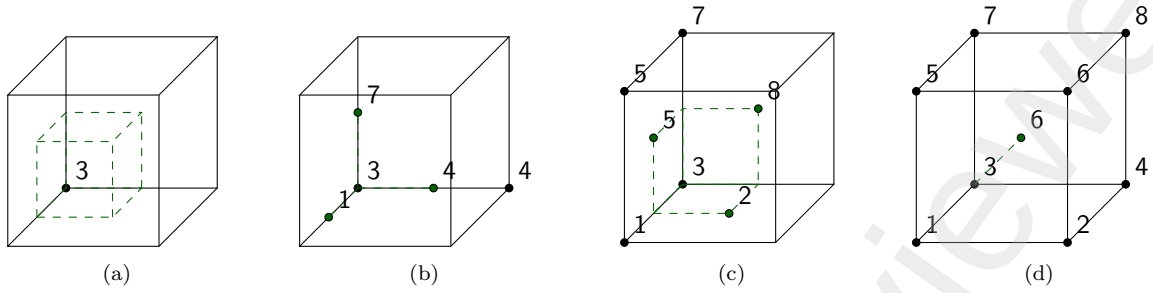


Figure 10: (a) illustration of the local numbering of a refined child cell (dashed green) from its common corner node in the parent local numbering (see Fig. 6), in this case corner node / child cell 3. Building of the child cell nodes from the parent cell (b-d) for Alg. 1: (b) edge-hanging nodes in the parent cell, built from the edges incident on the shared corner node, (c) face-hanging nodes in the parent cell, built from the faces incident on the shared corner node, and (d) diagonally opposed node, built from the 8 corner nodes of the parent cell (which may itself possess more than 8 nodes, see e.g. Fig 4). The green nodes in (b) and (c) may already be present in the parent cell. The center node in (d) is always new.



Figure 11: Illustration of (a) cells being coarsened not being a tessellation of the resulting coarsened cell (b). This situation only occurs in Lagrange-AMR and ALE-AMR situations, and results from the coarsened cells not having enough nodes to match the outgoing geometry. The green grid pattern highlights the geometrical region that breaks the tessellation.

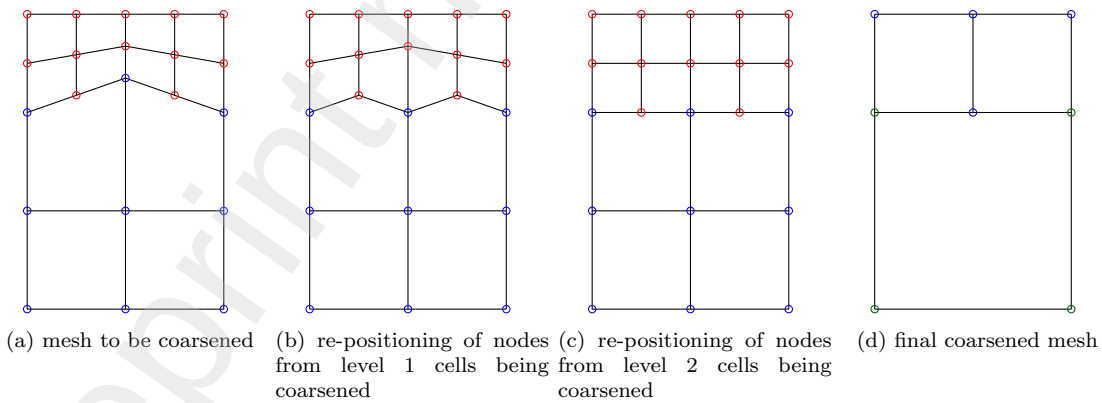


Figure 12: Illustration of Lagrangian and ALE remapping for cells being coarsened. Starting from an initial mesh (a), nodes of cells flagged for coarsening at increasing refinement depth are re-positioned (a to b and b to c). An ALE face-sweep (Sec 3.1) used to remap from (a) to (c), and the AMR remapping (Sec 5.2) from (c) to (d). Green, blue and red nodes indicate octant corner nodes belonging to levels 0, 1 and 2, respectively.



Figure 13: Illustration of cell tangling that can occur during coarsening. The blue cells in (a) are being coarsened, resulting in a tangled cell in the ingoing mesh (b). This situation only occurs in Lagrange-AMR and ALE-AMR situations.

5.1.1. Since cells at each increasing refinement depth level depend on node positions established at the previous depth level, this node repositioning must be performed iteratively for increasing refinement levels. An illustration of the repositioned nodes is given in Fig. 12. The node re-positioning algorithm is based on the known corner numbering for the octants. The MPI decomposition introduces some difficulty because the ghost layer is not wide enough to contain all external outgoing cells undergoing coarsening. As such, the position of duplicated internal nodes can only be determined from MPI ranks that own a group of 8 cells undergoing coarsening and that are incident on that node. Since that rank may not be the owner of the duplicated internal node (see Sec. 4.2.2), specific handling is required. A simplified description of the corresponding coarsening algorithm is provided in Alg. 2.

Algorithm 2 Algorithm for re-positioning nodes prior to coarsening.

```

for  $l \in [1, l_{\max}]$  do
  for  $d \in \mathcal{C}_{\text{int}}^{\text{in}}$  do
    if  $d$  is a coarsened cell then
      Obtain the corner nodes positions from the ancestors of  $d$  in the outgoing mesh; local corner
      node  $i \in [1, 8]$  of  $d$  is also the local corner  $i$  in the  $i$ -th element of  $\mathcal{A}_d$  (see Fig. 10 (a)). These nodes will
      be the corner nodes of the ingoing cell  $c$ .
      The node in the outgoing mesh that corresponds to the barycenter of  $d$  is the local node opposed
      to local node 1 (i.e. 8) in ancestor 1 of  $\mathcal{A}_d$  (see Fig. 6 (a)). Set its position from the barycenter of the
      corner nodes of  $c$ .
      The 6 nodes in the outgoing mesh that correspond to the 6 face barycenters of  $d$  are found using
      the mapping relations for diagonally opposed corner nodes in the outgoing cells (Fig. 10 (c)). Similarly
      the 12 nodes in the outgoing mesh that corresponds to the 12 edge barycenters of  $d$  are found using the
      mapping for edge-opposed nodes (Fig. 10 (b)). Set these 18 node positions from the corner nodes of  $c$ .
    end if
  end for
  Loop over the owned cells in the MPI communicator and set the 8 corner node positions for each cell,
  alongside a logical flag indicating if that node was reset in that cell and that MPI rank
  Synchronize the above structure
  Loop over the cells in the external layer and set the position of internal nodes to that of the remote,
  only if that remote rank reset the node position in that cell
end for

```

Finally, it should be noted that the coarsening operation itself may introduce tangled cells. This can happen when cells in the outgoing mesh are non-convex, or in some configurations as illustrated in Fig. 13. As such, this intermediate ALE remapping step is iterated with disentangling steps to ensure mesh validity prior to the AMR remapping. The disentangling algorithm is based on a line-sweep method and described in the Part II of this study.

5.2. AMR Remapping on preprocessed mesh

We now describe the remapping of quantities from the outgoing mesh after preprocessing (Secs. 5.1.1 and 5.1.2) to the ingoing mesh. In the ALE case, we have described in Sec. 3.1 a swept-face algorithm that performs the remapping of quantities between two meshes with the same topology. In the case of regridding, the cells of the outgoing and ingoing meshes have different topologies but have been arranged such that they are tessellations of one another. The remapping step then proceeds similarly to the ALE case, except that face-fluxes are replaced by integrals over the cells themselves. We leverage the 3D multimaterial ALE algorithms presented in Sec. 3.1.

We recall that quantities denoted with a subscripts c and d denote the outgoing and ingoing meshes, respectively. The AMR remapping is written on the same set of conserved quantities ψ^k as for the ALE: partial masses m^k , partial internal energies $\mathcal{E}^k$, partial momentum $m^k \mathbf{V}$, and partial volumes V^k . As for the ALE, the first step is to define the material planes Π^k from partial volumes V_k and the gradients in α^k . By intersecting Π^k with the mesh cells, we define the material polytopes ω_c^k for each material k in each outgoing cell c . Using the framework of Sec. 3.1, we define the partial quantities on the ingoing mesh as

$$\begin{aligned} m_d^k &= \sum_{c \in \mathcal{A}_d} \int_{\omega_{dc}^k} \rho^k d\mathbf{x}, & m_d &= \sum_k m_d^k, \\ \mathbf{V}_d &= \sum_k \left[\frac{1}{m_d^k} \sum_{c \in \mathcal{A}_d} \int_{\omega_{dc}^k} \rho^k \mathbf{V} d\mathbf{x} \right], \\ \varepsilon_d^k &= \frac{1}{m_d^k} \sum_{c \in \mathcal{A}_d} \int_{\omega_{dc}^k} \rho^k \varepsilon^k d\mathbf{x}, \\ \alpha_d^k &= \frac{1}{V_d} \int_{\omega_{dc}^k} d\mathbf{x}, \end{aligned} \quad (24)$$

where the volume integral is carried over material polytope $\omega_{dc}^k = \omega_d \cap \omega_c^k$ defined by the intersection between the ingoing cell polygon ω_d and the material polytope ω_c^k (see Fig. 15). The ancestor list $\mathcal{A}_d$ contains only one element for refinement, and 8 for coarsening. Eqs (24) are integrated at second-order using the same procedure detailed in Sec. 3.1; namely using Eq. (11) with $\bar{\Omega}_{p,p^+,p_f}^k$ swapped for ω_{dc}^k . Gradients are computed on the outgoing mesh using a least-square evaluated on the nodal stencil, and a Barth-Jespersion slope limitation evaluated at the nodes of the cell. For refinement, Eq. (11) is equivalent to interpolating the volumetric conserved variables $\hat{\psi}^k$ at the centroids of the 8 refined cells. By virtue of the slope-limitation of the gradient on the outgoing mesh, the remapped quantities cannot undershoot or overshoot the local minima/maxima evaluated on the nodal stencil of the outgoing mesh. As such, the projection also ensures positivity of mass, momentum and internal energy on the ingoing mesh. For coarsening, Eq. (11) can be further simplified by noting that in that case $\mathbf{C}_c^k = \mathbf{C}_{dc}^k$, such that all terms involving the gradient cancel out and we simply obtain

$$\begin{aligned} m_d^k &= \sum_{c \in \mathcal{A}_d} m_c^k, & m_d &= \sum_k m_d^k, \\ \mathbf{V}_d &= \frac{1}{m_d} \sum_{c \in \mathcal{A}_d} \mathbf{P}_c, & \varepsilon_d^k &= \frac{1}{m_d^k} \sum_{c \in \mathcal{A}_d} \mathcal{E}_c^k. \end{aligned} \quad (25)$$

The remapped quantities chosen in the AMR step ensure the conservation of partial internal energy, partial momentum and partial mass. To ensure total energy conservation, we apply the same method as in Sec. 3.1 and correct the internal energy by remapping the kinetic energy $\mathcal{E}_{d,\text{kin}}^k$ and comparing with the remapped

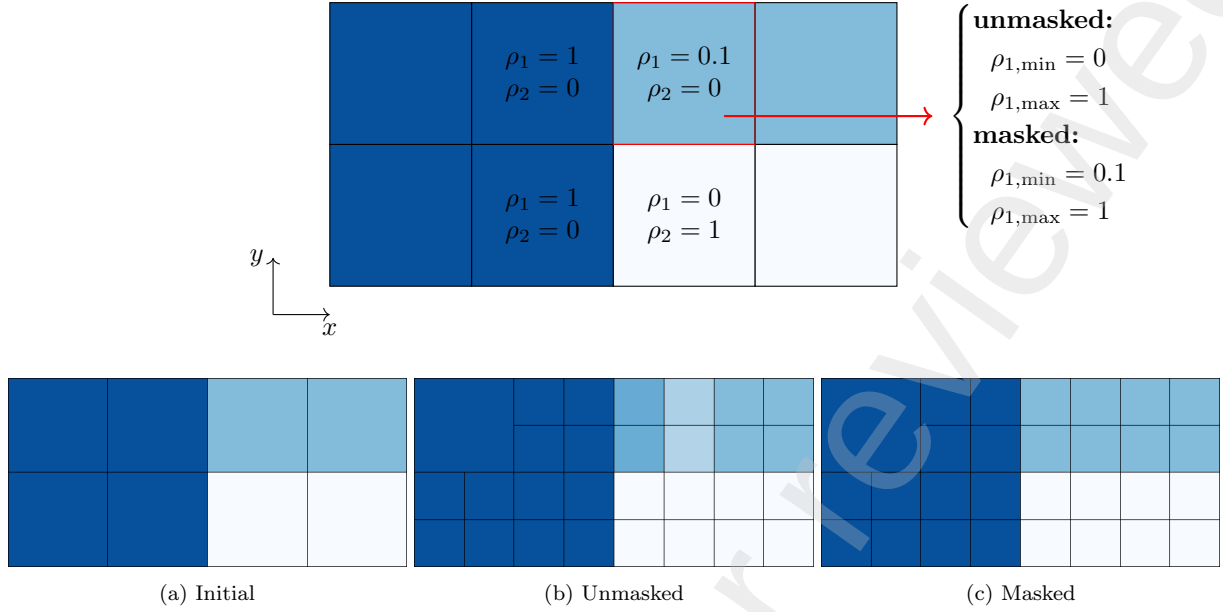


Figure 14: (top) schematic illustration of oscillation issues that can arise at triple-point-like configurations. The material 1 (blue) surrounds material 2 (white) around a node. The bounding values for the second-order limitation procedure for the red cells are shown on the right, for the unmasked and the masked procedures, respectively. (a,b,c) density of material 1 in the same setup, showing (a) the initial state, (b) the density after refinement when using unmasked gradients and limitation, where the limitation stencil allows for an undershoot, and (c) the corresponding density with masked gradients and limitation, where the limiter corrects the projection to first-order.

momentum

$$\begin{aligned}\mathcal{E}_{d,\text{corr}}^k &= \mathcal{E}_d^k + \left(\mathcal{E}_{d,\text{kin}}^k - \frac{1}{2} \frac{\mathbf{P}_d^k \cdot \mathbf{P}_d^k}{m_d^k} \right), \\ \mathcal{E}_{d,\text{kin}}^k &= \frac{1}{2} \sum_{c \in \mathcal{A}_d} \int_{\omega_{dc}^k} \rho^k \mathbf{V} \cdot \mathbf{V} d\mathbf{x}, \\ \mathbf{P}_d^k &= \sum_{c \in \mathcal{A}_d} \int_{\omega_{dc}^k} \rho^k \mathbf{V} d\mathbf{x}.\end{aligned}\tag{26}$$

Remark. An important note must be made on the treatment of the cells which are empty with respect to a given material, here denoted by index k . Limited gradients can be computed as usual, by substituting the polytope centroid for the cell centroid and setting the partial quantities to zero. However, this approach can lead to oscillations in triple-point-like situations. This drawback is illustrated in Fig. 14. In that case, it is observed that the lower limit for the limitation would be $\rho_{k,\min} = 0$ due to the neighbor empty cell. This in turn would allow the gradient to undershoot on the side of the mesh where $\rho_k > 0$, leading to an oscillation along the x-direction. Similarly, it is arguably not clear if the gradient itself should have a component in the y-direction in that case, which is only introduced by the presence of the empty cell for material k . To address this issue, in this paper, we choose to *mask* cells in the computation of gradients and their limitation. Namely, when computing $(\nabla \hat{\psi}^k)$, we exclude all cells with $\alpha^k < 10^{-5}$ for the least-square procedure and for the determination of $\hat{\psi}_{\min}^k$ and $\hat{\psi}_{\max}^k$ for the second-order limitation procedure. A comparison of AMR at such a triple point configuration between the unmasked and masked operators is shown in Fig. 14(b,c).

5.3. AMR criteria

A large variety of AMR refinement/coarsening criteria can be formulated depending on the problems being studied. In an Eulerian context, criteria are often based on absolute differences between adjacent

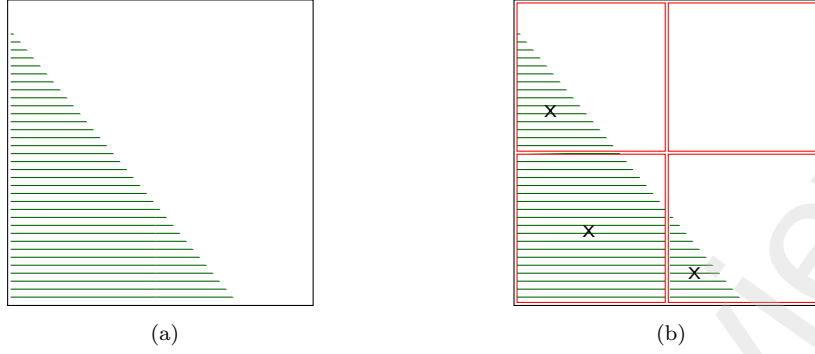


Figure 15: Handling of multi-material interfaces; (left-to-right) refinement of 1 outgoing cell into 8 ingoing cells (4 in this 2D illustration), (right-to-left) coarsening of the outgoing cells into an ingoing cell. On the left picture a mixed cell is defined by a black boundary, with two materials (green and white) separated by a planar interface. The intersections between ingoing cells (right) and the planar interface define material polytopes, which centroids are used for the second-order projection (see e.g. black crosses for the centroid of green material in the red cells).

cells. As an example, an Eulerian AMR criterion aiming to resolve shock propagation can be based on the absolute density difference between a cell and its neighbor. It is straightforward to see that absolute differences criteria are less adapted to the Lagrangian formalism, where the mesh cells following the flow will naturally lead to sharp discontinuities (e.g. shocks, thermal fronts) which do not require refinement.

Another class of AMR criteria relies on the estimation of local error by comparing the results obtained from a larger timestep on a coarser mesh with two steps on the current mesh. This particular approach is called Richardson extrapolation and is notably used in CLAWPACK [42]. Its interest lies in its generality and applicability to different problems. The main drawback is the added code complexity to handle this two-step approach. Finally, some AMR criteria can be formulated based on non-hydrodynamic processes, i.e. to adapt resolution for physics solvers coupled to the hydrodynamics.

5.3.1. General form of the refinement/coarsening threshold

In this paper, we consider two normalized refinement criteria denoted $\mathcal{A}_{\text{diff}}$ and $\mathcal{A}_{\text{o2}}$ (given in what follows). Both criteria are computed from the same set of quantities: total pressure p , material density ρ , partial volume fraction α^k and velocity vector $\mathbf{v}$. We define a metric value μ which controls refinement and coarsening through the following conditions:

$$\text{refinement if : } \mu_c > T \quad \text{and} \quad \max_{c' \in \mathcal{C}(\mathcal{P}(c))} |\mathbf{v}_{c'}|/a_{c'} > 10^{-6}, \quad (27)$$

$$\text{coarsening if : } \mu_c < \epsilon_T T \quad (28)$$

where parameter T is a scalar threshold and the coarsening threshold is set to $T\epsilon_T$ with $\epsilon_T < 1$.

5.3.2. AMR criteria

The first criteria, $\mathcal{A}_{\text{diff}}$, considers the RMS of the absolute difference between the current cell values and those on the nodal neighborhood

$$\begin{aligned} \mathcal{A}_{c,\text{diff}}^\phi &= \frac{\sqrt{\frac{1}{\text{card}(\mathcal{C}(\mathcal{P}(c)))} \sum_{c' \in \mathcal{C}(\mathcal{P}(c))} (\phi_{c'} - \phi_c)^2}}{\max(\phi_{\text{ref}}, \phi_c)}, \\ \mathcal{A}_{c,\text{diff}}^\phi &= \frac{\sqrt{\frac{1}{\text{card}(\mathcal{C}(\mathcal{P}(c)))} \sum_{c' \in \mathcal{C}(\mathcal{P}(c))} \left((\phi_{c'} - \phi_c) \cdot \frac{\phi_c}{|\phi_c|} \right)^2}}{\max(|\phi_{\text{ref}}|, |\phi_c|)}, \end{aligned} \quad (29)$$

where ϕ and ϕ denote a scalar and a vector, respectively, $\mathcal{C}(\mathcal{P}(c))$ denotes the set of cells sharing a node with c , and ϕ_{ref} denotes a reference value.

The metric value $\mu_{c,\text{diff}}$ then reads:

$$\mu_{c,\text{diff}} = L_2(\mathcal{A}_{\text{diff}}^P, \max(\mathcal{A}_{\text{diff}}^\rho, \mathcal{A}_{\text{diff}}^{\alpha^k}), \mathcal{A}_{\text{diff}}^{\mathbf{v}}) \quad (30)$$

where L_2 denotes the L_2 norm $L_2(x_i) = \sqrt{\sum_i x_i^2}$. Note that the contributions from the density and material volume fractions have been grouped into one so that the criteria do not depend on the number of materials.

The second criterion, denoted $\mathcal{A}_{\text{o2}}$, aims to quantify second-order reconstruction errors in the numerical scheme. We start by defining the RMS residual of the difference between cell values and their second-order reconstruction evaluated on the node stencil:

$$\begin{aligned} \mathcal{A}_{c,\text{o2}}^\phi(\nabla\phi) &= \frac{\sqrt{\frac{1}{\text{card}(\mathcal{C}(\mathcal{P}(c)))} \sum_{c' \in \mathcal{C}(\mathcal{P}(c))} (\phi_{c'} + (\mathbf{r}_{c'} - \mathbf{r}_c) \cdot \nabla\phi_c - \phi_c)^2}}{\max(\phi_{\text{ref}}, \phi_c)}, \\ \mathcal{A}_{c,\text{o2}}^\phi(\nabla\phi) &= \frac{\sqrt{\frac{1}{\text{card}(\mathcal{C}(\mathcal{P}(c)))} \sum_{c' \in \mathcal{C}(\mathcal{P}(c))} \left((\phi_{c'} + (\mathbf{r}_{c'} - \mathbf{r}_c) \times \nabla\phi_c - \phi_c) \cdot \frac{\phi_c}{|\phi_c|} \right)^2}}{\max(|\phi_{\text{ref}}|, |\phi_c|)}, \end{aligned} \quad (31)$$

where the criterion depends on the definition of the gradient operator $\nabla\phi$. There is a caveat that the hydrodynamics uses a second-order reconstruction that is different from that of the ALE and AMR. Namely, the hydrodynamics uses a discretization of the Green operator (see Sec. 2.3 and Ref. [30]), while the AMR and ALE remapping steps use least-square gradients constructed from the nodal stencil. In addition, since the AMR step is executed after the ALE rezoning/remapping, the gradients computed after the ALE do not necessarily accurately reflect the second-order error from the hydrodynamics step. As such, this criterion is formulated to combine error estimates from multiple sources: from the hydrodynamics gradients denoted $\nabla\phi_{\text{hyd}}$, and from the least-square gradients constructed after the ALE step and denoted $\nabla\phi_{\text{amr}}$. Although this approach may appear complex, in our tests, it has given the best results relating to preserving symmetries in the flow. The combined metric value for the error estimate criteria then reads:

$$\begin{aligned} \mu_{c,\text{o2}} &= \\ &\left(\max[\mathcal{A}_{\text{o2}}^\rho(\nabla\phi_{\text{amr}}), \mathcal{A}_{\text{o2}}^{\alpha^k}(\nabla\phi_{\text{amr}})]^2 + \max[\mathcal{A}_{\text{o2}}^P(\nabla\phi_{\text{amr}})^2 + \mathcal{A}_{\text{o2}}^{\mathbf{v}}(\nabla\phi_{\text{amr}})^2, \mathcal{A}_{\text{o2}}^P(\nabla\phi_{\text{hyd}})^2 + \mathcal{A}_{\text{o2}}^{\mathbf{v}}(\nabla\phi_{\text{hyd}})^2] \right)^{1/2} \end{aligned} \quad (32)$$

In both cases, the error on the vectors (here, velocity) is computed on the projection of the extrapolated or neighbor values with the cell value. This is done for consistency with the velocity limitation method of the hydrodynamics solver (see Sec. 2.3), and was found to provide more refinement patterns preserving symmetry.

5.3.3. Choice of the refinement/coarsening ratio

For both criteria, the refinement threshold T and the coarsening threshold $T\epsilon_T$ must be set. To avoid introducing too many free parameters, we can set ϵ_T based on theoretical considerations. For the trivial case of a constant profile of quantity ϕ , both $\mathcal{A}_{\text{diff}}$ and $\mathcal{A}_{\text{o2}}$ are zero. Assuming a 1D linear profile $\phi = ax$ and cells spaced by dx , the normalized absolute difference $\mathcal{A}_{\text{diff}}$ scales as dx/x . A local cell refinement ($dx \rightarrow dx/2$) then decreases $\mathcal{A}_{\text{diff}}$ by 2. For a profile of the form $\phi = ax^2$, the decrease in $\mathcal{A}_{\text{diff}}$ is in the range $[2 - 4]$ depending on the value of dx/x . For the second-order reconstruction error $\mathcal{A}_{\text{o2}}$, the error in the linear profile is zero by definition. The error for a quadratic profile scales as $dx^2/2$, such that a local refinement lowers $\mathcal{A}_{\text{o2}}$ by a factor of 4. From these considerations, we choose $\epsilon_T = 1/5$, that is, 80% of the theoretical maximum value of $1/4$ described above. The extra leeway is taken to avoid successive refinement and coarsening close to threshold.

5.4. Summary operation diagram

A summary flowchart of the interfacing between the hydrodynamics and *p4est* for AMR operations is given in Fig. 16.

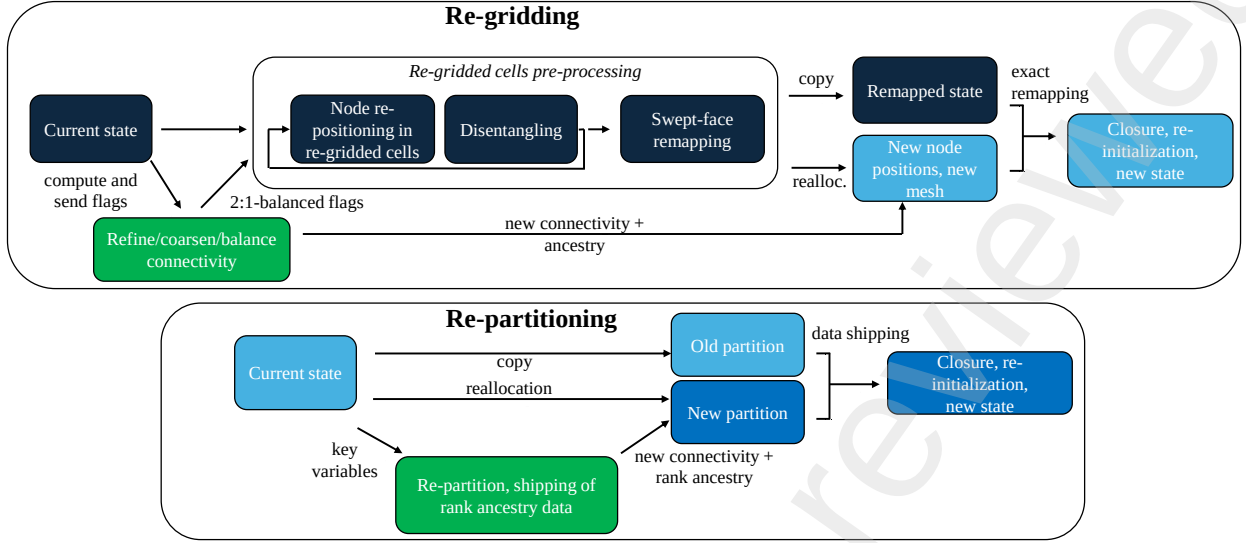


Figure 16: Schematic summary of the regridding and repartitioning operations. Green boxes refer to operations carried out by *p4est*. The regridding and repartitioning steps are carried sequentially to ensure that groups of 8 octants belonging to the same ancestor are owned by the same MPI rank.

Re-gridding: the AMR procedure is called after the remapping step. The flagging functions (Sect. 5.3) are applied, set in *p4est* memory, and AMR is carried out on the topology. Using the 2:1 corner-balanced AMR flags, the outgoing mesh is preprocessed by performing an intermediate ALE step for refinement (Sec. 5.1.1) and for coarsening (Sec. 5.1.2 and Alg. 2). This preprocessing is iterated with disentangler steps to ensure that nonconvex cells are properly handled. Then, the outgoing mesh geometry is copied, along with variables required for the projection (see notably Sec 5.2). The new octant topology is used to reconstruct the polygonal mesh (see Sec. 4.2), and the new nodes are defined (Alg. 1 and Sec. (5.1.1)). Combining this new mesh with the hydrodynamics variables and geometry information from the outgoing mesh, the remapping is carried out (Sec. 5.2). The system is closed by calling the EOS and recomputing secondary variables.

Repartitioning: mesh repartitioning is carried out after each regridding step. To do so, we start by setting in *p4est* octant memory the current local cell number and MPI rank owner. *p4est* then performs the repartitioning (see Sec. 4.1.2) and handles the shipment of octants to new ranks. We recover a new topology and reconstruct a mesh (see Sec. 4.2). Using the shipped ancestry information, i.e. the previous owner and local cell number of each new cell, we communicate hydrodynamic data from the previously owned cells to their new owners. The system is then closed by calling the EOS and re-computing secondary variables.

Finally, it should be noted that the regridding and repartitioning steps are strided, and are typically called every ~ 10 hydrodynamic steps.

6. Numerical tests

We now present a series of test cases that show the accuracy and performance of the algorithms presented in this paper. We start by a discussion of numerical artifacts that can appear when propagating a shock in a mesh with uneven resolution in Sec. 6.1. We then explore results for the Sod shock tube in Sec. 6.2, interacting blast-wave problem in Sec. 6.3, Sedov explosion in Sec. 6.4, and multi-material triple point in Sec. 6.5. In this Part I paper, we only present results using either the Lagrange or (indirect)-Euler schemes (with and without AMR). However, we recall that the ALE steps are involved in the AMR part of the Lagrange-AMR 5. ALE-AMR results are presented specifically in Part II, including: ALE-AMR version of the triple-point, multi-material rotor, Rayleigh-Taylor single-mode growth, and ICF-like perturbed implosion.

6.1. Shock propagation in meshes with uneven resolution

We first start by pointing out a shortcoming of our scheme when dealing with shock propagation in meshes with strong changes in resolution. Such meshes will occur within the AMR framework (with factors of $\times 2$ or $/2$ introduced by refinement and coarsening), and, as shown here, care must be taken when refining/coarsening around shocks.

In Ref. [43], Noh reported two numerical artifacts denoted as type #1 error, which corresponds to the well-known *wall-heating* problem, and type #2 error, related to the characteristic length of artificial viscosity and occurring for shocks propagating in uneven meshes. For large jumps in mesh resolution, the viscosity of the scheme will jump accordingly, which can lead to oscillation in the solution for density in a similar manner as was noted in Fig 2 of Ref. [43]. An illustration of this problem is given in Fig. 17, which consists of the Sod test (see Sec. 6.2 for the Sod test), ran on a mesh with a resolution jump of $2\times$ at $x = 0.6$ cm. A density discontinuity can be observed in the shocked region, which we believe is related to the type #2 error reported by Noh, but this may require additional investigation.

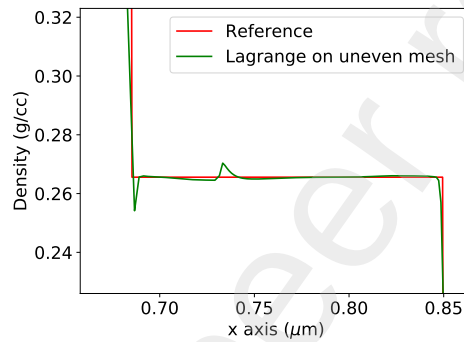


Figure 17: Lagrange Sod test result in the shocked region, for a mesh with a $2\times$ discontinuity in initial resolution. The undershoot on the left side is due to wall-heating and also present on regular meshes. The up-shoot around $x=0.74$ cm is due to the uneven mesh resolution and likely caused by a variation in the viscosity of the scheme.

Nevertheless, this behavior highlights an important requirement for the AMR scheme: care must be taken that shocks travel in regions of the same refinement level. These requirements are satisfied by our refinement criteria given in Sec. 5.3, however, there are also practical consequences: (i) calls to AMR refinement/coarsening should not be strided too much, to avoid shocks leaving refined regions, and (ii) a buffer region of refined cells can be added around cells flagged for refinement, for safety. In this paper, a 1-cell-wide buffer of refined cells is flagged around cells flagged for refinement.

6.2. Sod shock-tube

The Sod shock tube test considers the evolution of a rarefaction wave and a shock wave created by an initial contact discontinuity. The initial conditions are: two equal subregions in a $0.01 \times 0.01 \times 1$ domain with density of 1 and pressure of 1 on the left side and density of 0.125 and pressure of 0.1 on the right side. In both region we set $\gamma = 1.4$. While the Sod test has a 1D geometry, our AMR implementation is purely 3D and leads to a 3D mesh. The non-AMR tests are run on a $4 \times 4 \times 200$ grid. The AMR tests are initialized on a $1 \times 1 \times 50$ grid, and a refinement depth of a maximum of 2 is set such that at maximum uniform refinement the two grid resolutions coincide. The hydrodynamic state is compared at $t = 0.2$ with the analytical solution. All 3D results are overlapped along the 1D direction.

We first benchmark the accuracy of the Lagrange scheme against the analytical solution (AMR is not used here). The results comparing the first-order and second-order solutions are given in Fig. 18. As expected, the second-order results better capture the analytical solution, but a slight undershoot remains present at the foot of the shocked region (wall-heating problem [43]).

A comparison of the Lagrange case with an Euler case (in this case the projection step is performed at each time step after the Lagrangian one, so-called *indirect-Euler*) is shown in Fig. 19. All cases are ran

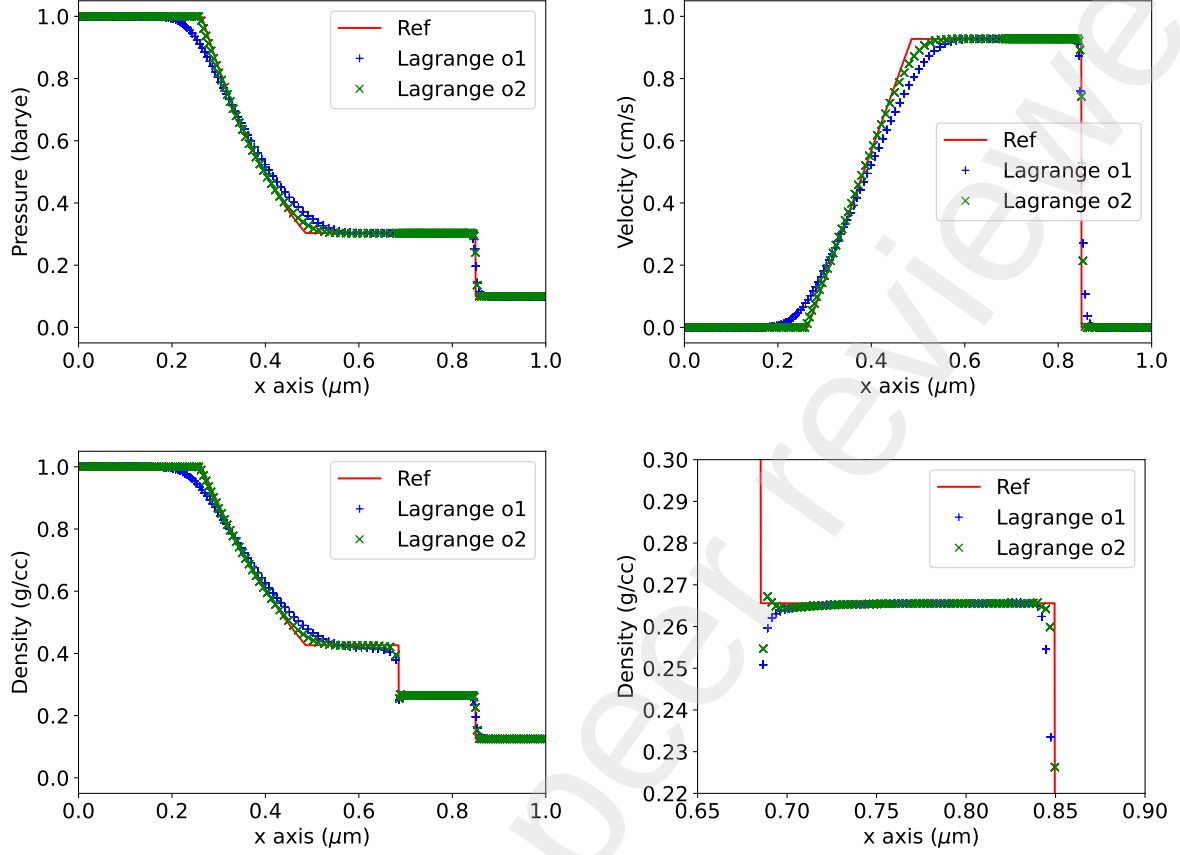


Figure 18: Pure Lagrangian results for the Sod shock tube, comparing the first-order (blue) and second-order (green) with the analytical solution (red). The panels show (top-left) pressure, (top-right) velocity, (bottom-left) density and (bottom-right) a close-up of the density behind the shock.

at second order for the hydrodynamics. The Eulerian solutions are compared using first-order and second-order remapping. The first-order remapping is found to be significantly more diffusive. Even at second-order remapping, the Eulerian solution is not as accurate as the Lagrangian case around the contact discontinuity. Note that both cases are run with the same initial resolution of 200 cells in the x-direction, as expected.

We now explore the sensitivity of the simulation results with our AMR parameters. We perform simulations initialized on a $4\times$ coarser grid with AMR enabled. The refinement parameter is varied in the range $T \in [0.01, 5]$. We recall that $\mathcal{A}_{o2}$ represents a normalized second-order reconstruction error and $\mathcal{A}_{diff}$ is a normalized absolute difference. The AMR is strided every 10 steps and we verified that the results are the same as without striding. We start by assessing the error in the final density profile between the analytical solution and the simulations. To that end, the simulated density profile is interpolated on a high-resolution grid corresponding to the analytical solution, and the error is defined as the RMS norm between the two (we recall that the RMS variation of x_i with respect to x_{ref} is $[(1/N) \sum_{i=1,N} (x_i - x_{ref})^2]^{1/2}$).

The scaling of the error versus the threshold value T is given in Fig. 20(a,b). This scaling provides an estimate of what is an adequate value for T for each criteria and how rapidly these reduce the error. In the Lagrange-AMR case, asymptotic values are reached around $T = 0.5$ for $\mathcal{A}_{diff}$ and $T = 0.1$ for $\mathcal{A}_{o2}$. Given the definitions of our two thresholds, it is consistent that lower values should be set for T for the local error ($\mathcal{A}_{o2}$) than for the absolute difference ($\mathcal{A}_{diff}$) in order to reach a similar error. The Euler-AMR case produces overall higher errors and reaches asymptotic convergence for lower values of $T = 0.1$ for $\mathcal{A}_{diff}$ and

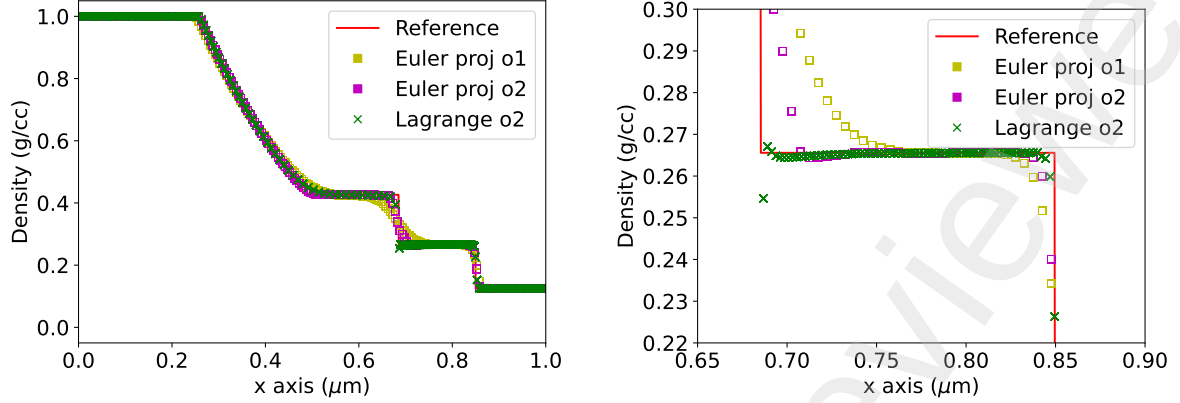


Figure 19: Density profiles for the Sod shock tube comparing the Lagrangian (green) and Eulerian (yellow, purple) results using the same initial mesh resolution. Both the Lagrangian and Eulerian cases use the second-order hydrodynamics scheme. The Eulerian results are shown with first-order remapping (yellow) and second-order remapping (purple).

$T = 0.05$ for $\mathcal{A}_{o2}$ are required. This more strict requirement may be related to the diffusive nature of the Euler case, which produces less sharp gradients.

The scaling of the error with T shows that the error decreases slightly more rapidly for $\mathcal{A}_{diff}$ than for $\mathcal{A}_{o2}$. However, this does not inform on how efficient these criteria are at decreasing the error, i.e. whether they are adding/removing cells in relevant regions and conservatively or generously. To that end, we now examine the scaling of the error as a function of the number of cells in the mesh, as shown in Fig. 20(c,d). In that case, an asymptotic error is reached with a similar number of cells in the Lagrange-AMR case and a lower number of cells with $\mathcal{A}_{o2}$ compared to $\mathcal{A}_{diff}$ in the Euler-AMR case. We also note that the criterion $\mathcal{A}_{diff}$ leads to larger mesh sizes when T keeps decreasing compared to $\mathcal{A}_{o2}$, which remains more conservative. This is the intended behavior, since $\mathcal{A}_{o2}$ will trigger the refinement only in regions of high second-order reconstruction error, whereas $\mathcal{A}_{diff}$ will trigger refinement in any sufficiently steep gradient.

We now examine the refinement depths reached in the mesh in various cases. Considering the Lagrange-AMR case, in Fig. 21 (a, b) we compare the two cases where the asymptotic error is first reached, $T = 0.5$ for $\mathcal{A}_{diff}$ and $T = 0.1$ for $\mathcal{A}_{o2}$. The $\mathcal{A}_{o2}$ case reaches maximum refinement around the contact discontinuities and the foot of the rarefaction wave. The criteria $\mathcal{A}_{diff}$ is more eager in the whole rarefaction wave region, since it is a region of strong gradients. We now consider the Euler-AMR case and compare in Fig. 21 (c, d) the two cases where asymptotic error is first reached, $T = 0.1$ for $\mathcal{A}_{diff}$ and $T = 0.05$ for $\mathcal{A}_{o2}$. Again, $\mathcal{A}_{diff}$ triggers refinement more eagerly. Comparing the Lagrange-AMR and Euler-AMR refinement pattern, the more diffusive aspect leads to broader refinement regions for $\mathcal{A}_{diff}$ (panel (c) vs (a)). For $\mathcal{A}_{o2}$ (panel (d) vs (b)) we observe spatial oscillations in the refinement pattern, but no oscillations are visible in the density profile. The oscillations are likely driven by slight deviations from the linear profile in the gradient region, which are picked up by the lower value of $T = 0.05$.

Close-ups of the density profiles for these four cases are shown in Fig. 22. The Lagrange-AMR simulations (panels (a) and (b)) using the two criteria exhibit similar accuracy, and are also close to the pure Lagrangian solution. The Euler-AMR cases (panels (c) and (d)) show a different behavior, with a slight loss of 1D in the shock. We believe this is due to an accuracy limitation in our least-square procedure for computing second-order reconstructions around domain boundaries in the ALE and AMR remapping algorithms. We also note that the instability identified in Sec. 6.1 is not observed, which indicates the the shock never leaves a refined region.

This first set of tests seems to indicate that both criteria are adequate for controlling the AMR behavior. The criterion $\mathcal{A}_{o2}$ is found to trigger refinement more conservatively and to produce accurate solutions. For the Sod test, values of T in the range of 0.01 to 0.05 yield good results for both the Euler-AMR and Lagrange-AMR cases. We now conduct a similar set of simulations for a different test case.

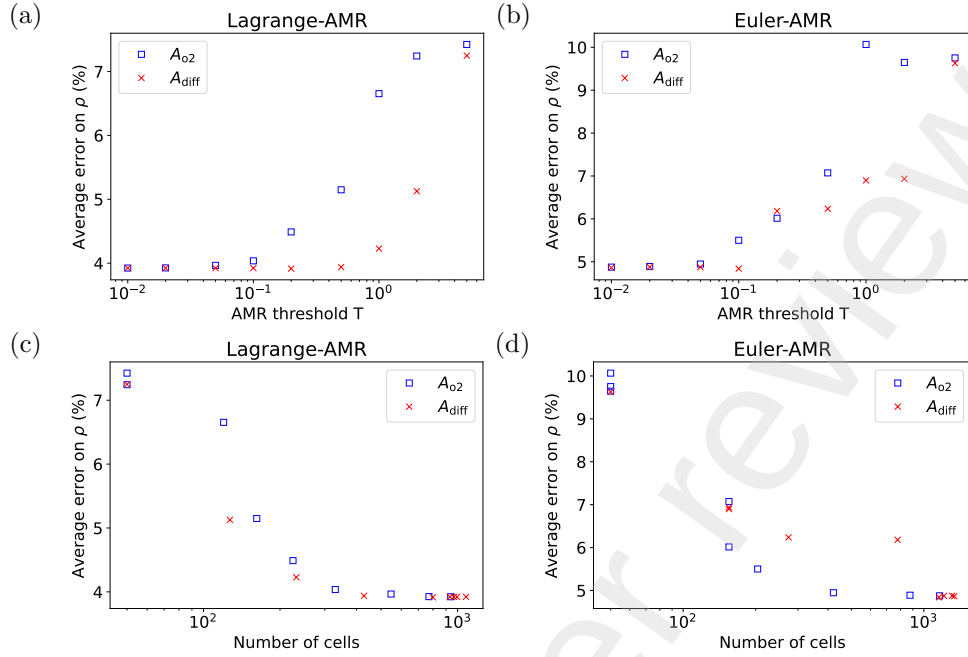


Figure 20: Comparison of the convergence rate of the error on final density profile for the Sod test, between (a,c) Lagrange-AMR and (b,d) Euler-AMR. The scaling is given as a function of threshold value T (a,b), and as a function of the number of mesh cells (c,d). The maximum refinement depth is 2 and the initial resolution is $(1 \times 1 \times 50)$. Note that the minimum error is highly sensitive to the accurate positioning of the contact discontinuities and is constrained by the initial resolution and maximum refinement depth.

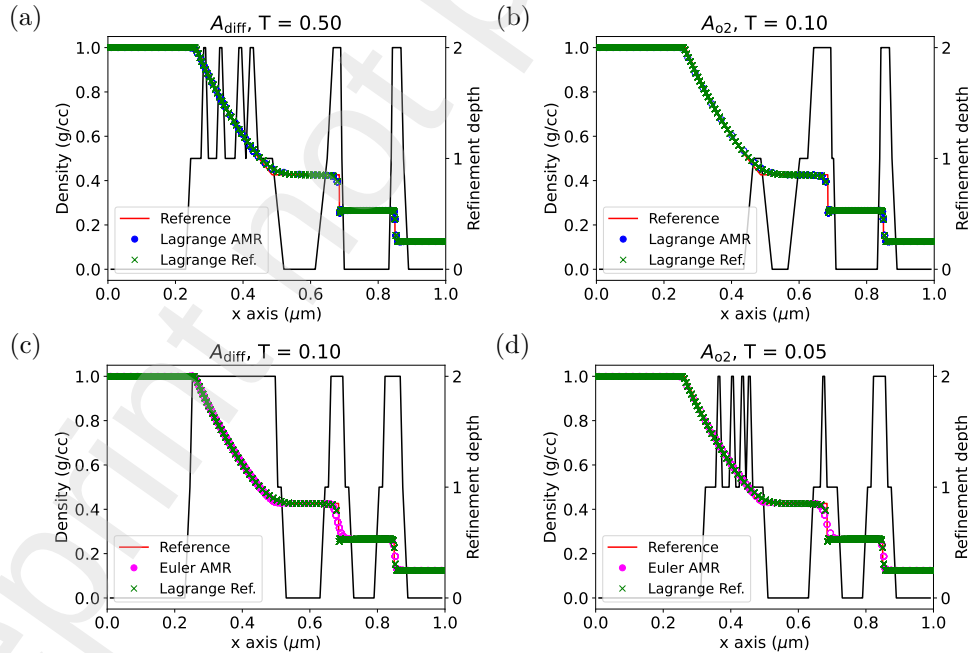


Figure 21: Density profiles for the Sod shock tube problem for Lagrange-AMR (a,b) and Euler-AMR (c,d), using the criterion $\mathcal{A}_{diff}$ (a,c) and $\mathcal{A}_{o2}$ (b,d). The threshold values are indicated on the figures, and correspond to the highest value where asymptotic error is reached. The reference solution is also shown in red and the pure Lagrange result in green. The black plain line indicates the refinement depth of the mesh as a function of position.

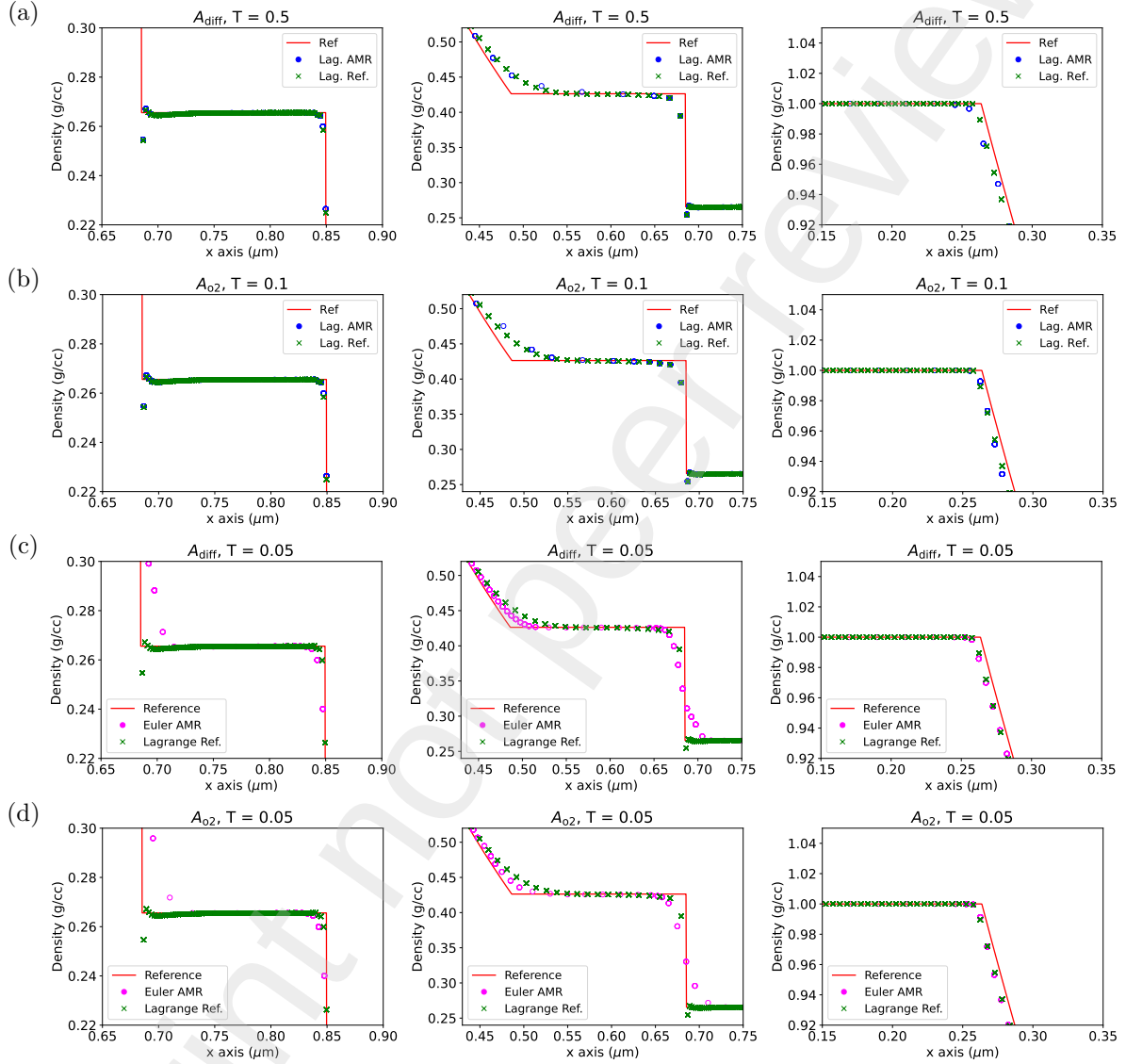


Figure 22: Close-up of the density profile for the Sod test problem, showing (left) the shock region, (middle) the central region and (right) the foot of the rarefaction wave region. Results are for (a,b) Lagrange-AMR and (c,d) Euler-AMR, using the criterion A_{diff} (a,c) and A_{O_2} (b,d). The threshold values are indicated on the figures, and are chosen slightly below or where the asymptotic error is reached. The reference solution is also shown in red and the pure Lagrange result in green.

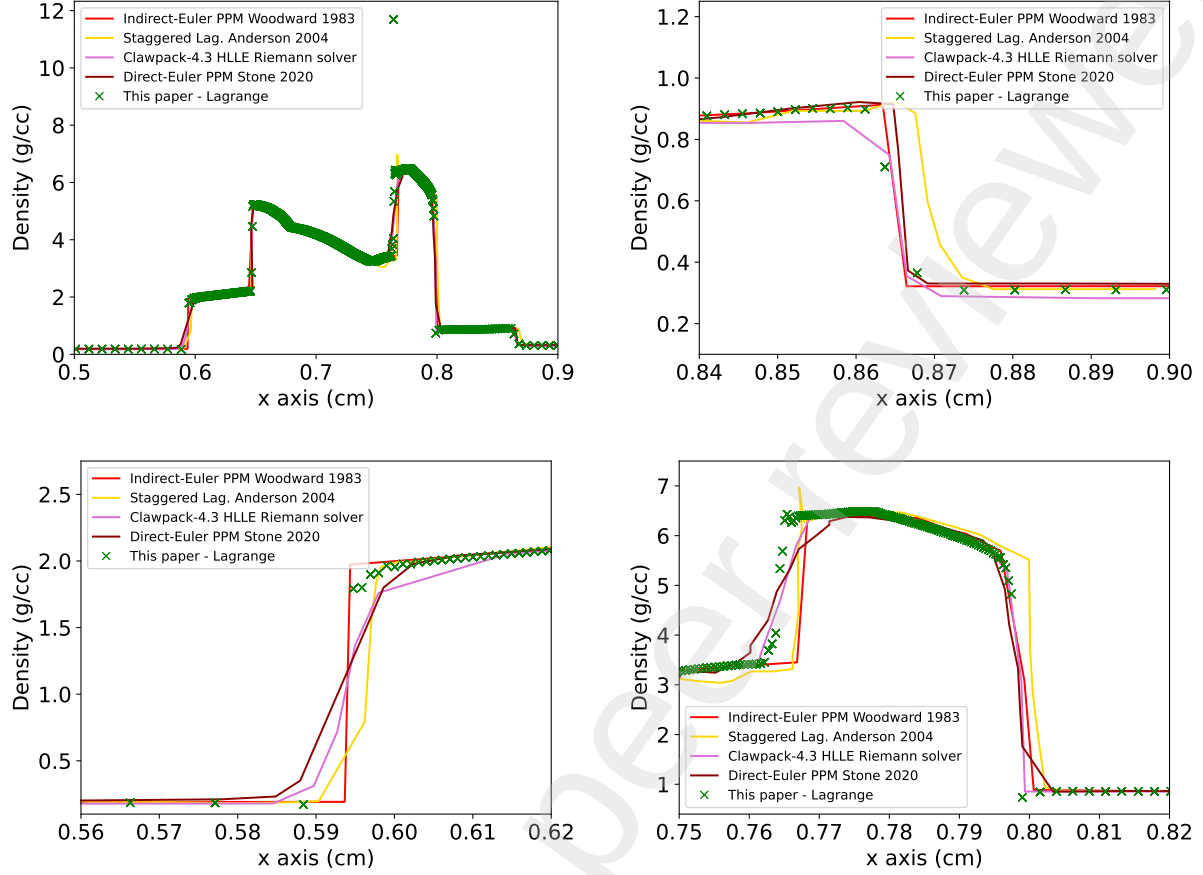


Figure 23: Comparison of the density profile at $t=0.038s$ for the interacting blast-wave problem with various references from the literature: (red) indirect-euler PPM from Ref. [44] with 3096 cells, (yellow) staggered Lagrangian result from Ref. [45] with 900 cells, (pink) HLLC Riemann Solver from Clawpack4.3 [46] with 2000 cells, (brown) direct-Euler PPM from Ref. [47] with 1200 cells, and (green) current work for pure-Lagrangian case with 500 cells.

6.3. Interacting blast-wave (Woodward-Collela test 1)

The interacting blast-wave problem [44] considers the evolution and interaction of two blast waves originating from two sides of a domain and initially separated by a low-density region. The three regions are initialized with pressures of 1000, 0.01 and 100, respectively. The domain is $0.01 \times 0.01 \times 1$ and the interfaces are positioned at 0.1 and 0.9. The density is initially uniform at 1 and the gas is an ideal gas with $\gamma = 1.4$. The calculations are compared at $t=0.038s$, shortly after the collision of the blast waves.

Since this test does not have an analytical solution, we start by comparing our results for a pure Lagrangian calculation to several solutions found in the literature. The comparison is shown in Fig. 23. Our calculation is setup with 500 cells, and compared to results for indirect-Euler PPM (3096 cells, Ref. [44]); staggered Lagrange (900 cells, Ref. [45]), HLLC Riemann Solver (2000 cells, Clawpack4.3, Ref. [46]), direct-Euler PPM (1200 cells, Ref. [47]). All results are in good agreement, with some differences in the sharpness and location of the discontinuities. Notably, our approach produces a sharp density peak around $x=0.764$, that is only seen in the staggered Lagrangian result to a lesser extent. In addition, the discontinuity at $x=0.764$ is offset with respect to that of Refs. [44, 45] by 0.005, while the location of that discontinuity for Refs [46, 47] is less clear, since the results are more diffusive. We also note that the position of the rightmost discontinuity is offset for the staggered Lagrangian result [45] compared to the other solutions.

Similarly to the Sod test, we explore the sensitivity of simulation results with our AMR parameters.

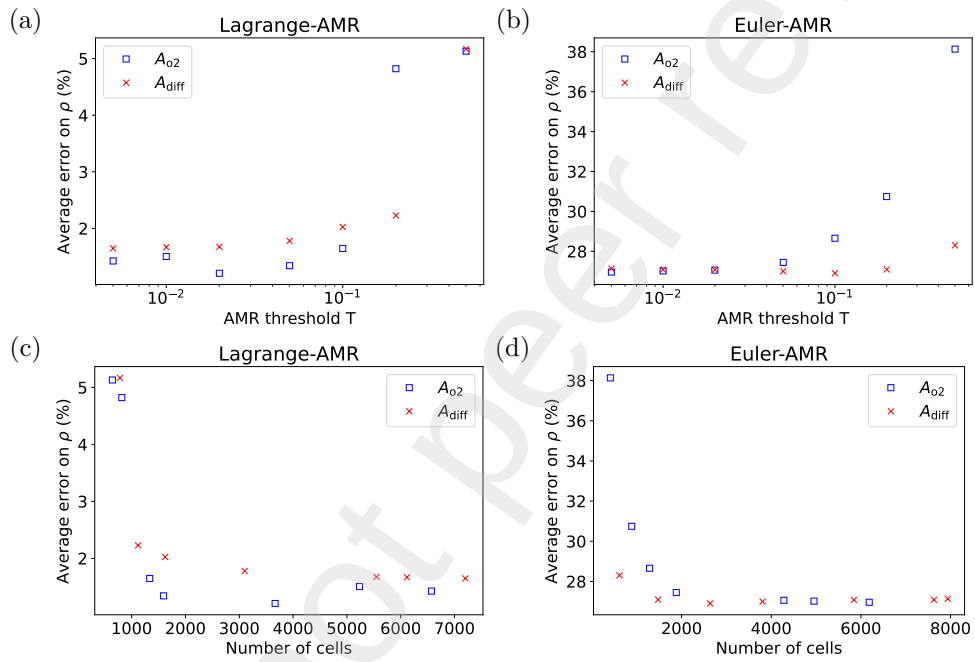


Figure 24: Comparison of the convergence rate of the error on final density profile for the interacting blast-wave problem, between (a,c) Lagrange-AMR and (b,d) Euler-AMR. The scaling is given as a function of threshold value T (a,b), and as a function of the number of mesh cells (c,d). The maximum refinement depth is 2 and the initial resolution is $(1 \times 1 \times 125)$. The error is computed with respect to a pure-Lagrangian simulation ran with $4 \times 4 \times 500$ cells. Note that the minimum error is highly sensitive to the accurate positioning of the contact discontinuities and is constrained by the initial resolution and maximum refinement depth.

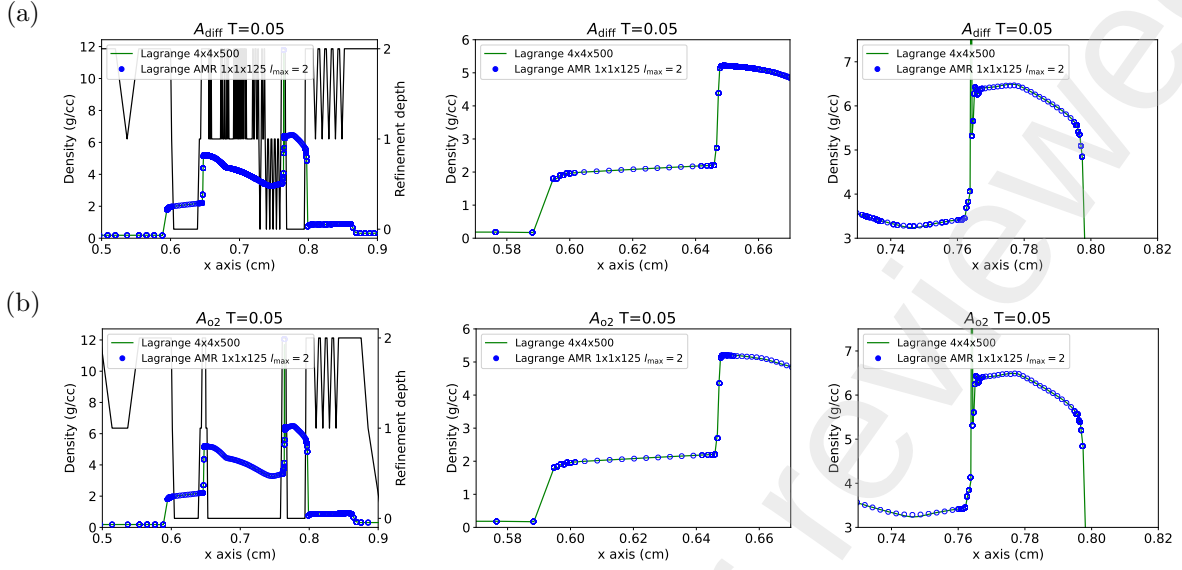


Figure 25: Comparison of the density profile for the interacting blast-wave problem at $t=0.038s$ between the pure Lagrangian case initialized on $4 \times 4 \times 500$ cells (green) and Lagrange-AMR calculations performed with: (a) $\mathcal{A}_{diff}$ with $T = 0.05$ and (b) $\mathcal{A}_{o2}$ with $T = 0.05$. These threshold values correspond to the highest values where asymptotic error is reached or nearly reached. AMR results are initialized on a $1 \times 1 \times 125$ grid with $l_{max} = 2$.

The AMR simulations are initialized on a $4 \times$ coarser grid of $1 \times 1 \times 250$ and a maximum refinement depth of 2 is set. The refinement threshold is varied in the range $T \in [0.005, 0.5]$ and the AMR is strided every 10 steps. Since there is no analytical solution here, the error is assessed by comparing with the pure Lagrangian solution obtained on the $4 \times 4 \times 500$ grid. Since the error metric is overly sensitive to the detailed position of the discontinuity around $x=0.764$, the region between $x=0.7635$ and $x=0.7645$ is excluded from the error calculation. The results are given in Fig. 24. In that setup, the asymptotic error is reached in the Lagrange-AMR case for $T \simeq 0.1$, both for $\mathcal{A}_{o2}$ and $\mathcal{A}_{diff}$. The error in the Euler-AMR case is much higher, as will be discussed later. The asymptotic error in that case is reached using $\mathcal{A}_{o2}$ for $T = 0.05$ and using $\mathcal{A}_{diff}$ for $T = 0.2$. Examining the scaling with the number of cells, we find that $\mathcal{A}_{o2}$ is in general more conservative than $\mathcal{A}_{diff}$.

The accuracy of the solution for Lagrange-AMR using the highest threshold value that reaches asymptotic error is shown in Fig. 25(a) for $\mathcal{A}_{diff}$ and (b) for $\mathcal{A}_{o2}$. Significant spatial oscillations of the refinement depth are observed in the $\mathcal{A}_{diff}$ case. These are associated with small-scale fluctuations in the density profile. Both cases are observed to remain 1D, but the results using $\mathcal{A}_{o2}$ exhibit fewer fluctuations in general. The Euler-AMR cases have significantly higher errors, associated with the diffusivity of the scheme that is unable to capture some of the sharp discontinuities. Euler-AMR results obtained with $\mathcal{A}_{diff}$ and $T = 0.05$ (below the asymptotic error value) are shown in Fig. 26. While the discrepancy with the Lagrange result is high, increasing the refinement depth allows to converge the solution, especially in the high density region.

The results in the interacting blast-wave problem have shown a similar behavior as in the Sod test problem; $\mathcal{A}_{diff}$ in general seem less adapted to the Lagrangian framework than $\mathcal{A}_{o2}$, as it tends to trigger refinement more eagerly while still exhibiting some oscillatory behavior in the solutions when the threshold value T is high. $\mathcal{A}_{o2}$ seem to produce more stable solutions with better efficiency in terms of mesh size. Compiling the results across Lagrange-AMR and Euler-AMR for the Sod and interacting blast-wave problems, a threshold value of $T \simeq 0.02$ for $\mathcal{A}_{o2}$ is adapted to our cases. This represents an L_2 average of the normalized second-order reconstruction error of 2% on the density, partial volume fractions, velocity projection, and pressure. Similarly, a value of $T \simeq 0.05$ for $\mathcal{A}_{diff}$ is adapted to our cases. This represents an L_2 average of the normalized absolute difference of 5% on density, partial volume fractions, velocity projection, and pressure.

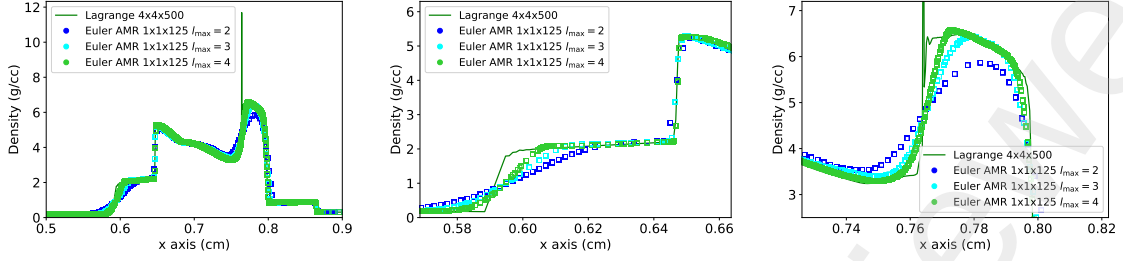


Figure 26: Comparison of the density profile at $t=0.038s$ between the pure Lagrangian case initialized on $4 \times 4 \times 500$ cells (green) and Euler-AMR results initialized on $1 \times 1 \times 125$ cells with $l_{\max} = 2$ (blue), $l_{\max} = 3$ (cyan) and $l_{\max} = 4$ (light green). The Euler-AMR results use the $\mathcal{A}_{\text{diff}}$ criterion with $T = 0.05$.

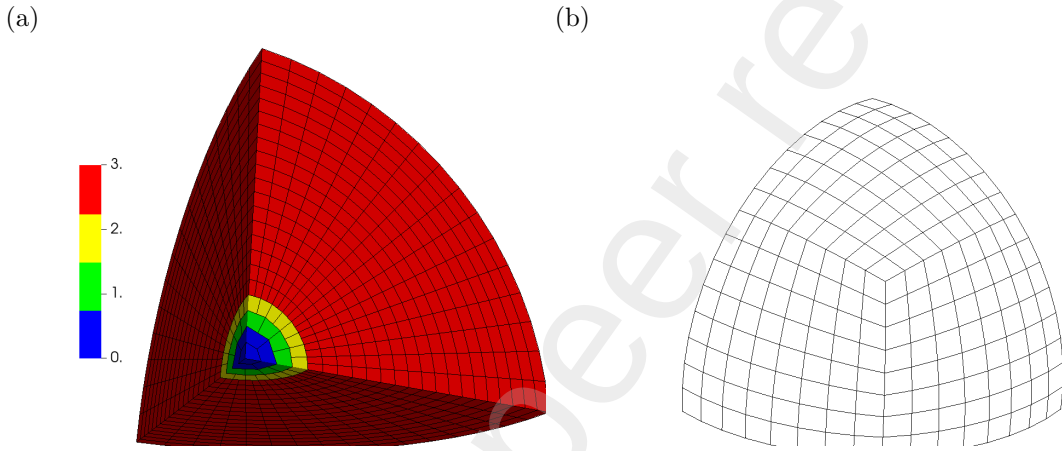


Figure 27: Initial meshes for the AMR Sedov test cases. The left panel shows the initial mesh refinement level imposed by construction of the mesh, which reaches a depth of $l = 3$. The right panel shows the arrangement of the cells in the angular direction.

6.4. Sedov explosion

The Sedov explosion case [48] considers the spherical expansion of a blast wave initialized from a spike in internal energy at the origin of the mesh. The gas is an ideal gas with $\gamma = 1.4$ initialized with a density of 1 and a pressure of 10^{-6} . The internal energy deposited at the origin is set to $\mathcal{E}^0 = 0.851072$ erg. Illustrations of EUCLHYD's second-order performances on cubic meshes for the Sedov test were presented in Refs. [30, 29, 4]. We consider the Sedov problem on the 8th sphere mesh and compare the pure Lagrangian, Lagrangian-AMR, and Euler-AMR results to the analytical solution from Ref. [49]. It should be noted that the 8th sphere mesh structure as presented in Fig. 3 is already refined to level 3 at $t = 0$ in order to obtain the desired angular resolution, as shown in Fig. 27(a).

The initial mesh resolution for the pure Lagrangian case is 256 cells per great circle (about 55×55 cells in the angular direction for an 8th of a sphere, see Fig. 27(b)) and 80 radial cells. The AMR cases are initialized with 64 cells per great circle and 20 radial cells, with a maximum refinement depth of $l_{\max} = 5$. Given the initial mesh refinement of 3, the relative refinement depth is 2, which corresponds at maximum refinement to the resolution of the pure Lagrange case. We consider two cases for each AMR simulation; $\mathcal{A}_{\text{o2}}$ with $T \simeq 0.02$ and $\mathcal{A}_{\text{diff}}$ with $T \simeq 0.05$, as informed by the tests performed in the Sod problem and the interacting blast wave problem. The final meshes are shown in Fig. 28.

The density profiles as a function of radius for the 6 cases are given in Fig. 29. We first note that all Lagrange-based solutions follow well the analytical solution, while all our indirect-Euler-based solutions exhibit a lower peak density related to an underresolved shock. This result is related to the choice of $l_{\max} = 2$, chosen to provide an equal maximum resolution with respect to the Lagrange case, but it is clear

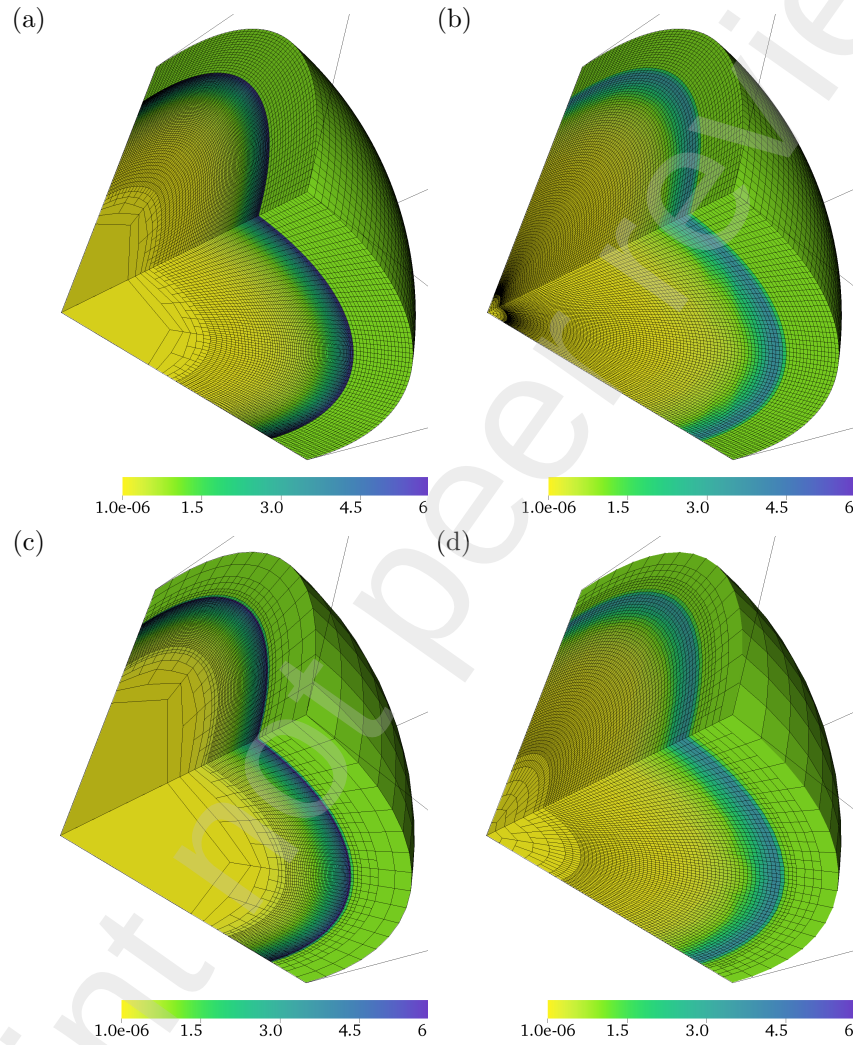


Figure 28: Final meshes for the Sedov test problem, for (a) Lagrange, (b) Euler, (c) Lagrange-AMR and (d) Euler-AMR. The color indicates the density. The AMR cases start with a $\times 4$ coarser initial resolution and the maximum refinement depth is set to 2. The criteria here is $\mathcal{A}_{\text{diff}}$ with $T = 0.05$.

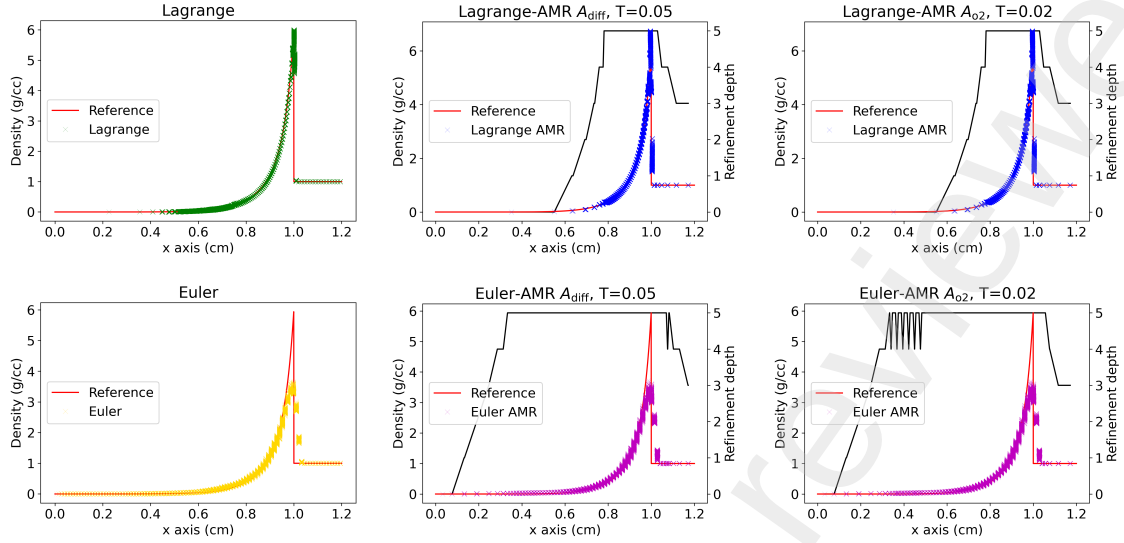


Figure 29: Comparison of the density profiles for the Sedov explosion case, for (green) pure Lagrange and (orange) pure Euler on $80 \times \{256/\text{great circle}\}$ mesh, (blue) Lagrange-AMR and (magenta) Euler-AMR, both initialized on a $20 \times \{64/\text{great circle}\}$ mesh with $l_{\max} = 2$. The criteria and threshold used are indicated in the panels and were chosen in Secs. 6.2 and 6.3. The black plain line indicates the refinement depth for the AMR cases, noting that the initial mesh is already refined to level 3 as shown in Fig. 27.

that a higher value for $l_{\max}$ is required in the Euler cases. In all cases, we do not see a significant difference in the refinement behavior between $\mathcal{A}_{o2}$ and $\mathcal{A}_{\text{diff}}$. Since our refinement criteria are based on local linear reconstructions and because the flow is spherical, it is expected that refinement is easily triggered in that case, such that both criteria yield equivalent patterns. Despite this, the AMR case remains attractive for its faster runtime. Defining the pure Lagrange solution has having a runtime of $\tau = 1$, we find the following runtimes for the other cases (all cases are run on 72 MPI ranks): Lagrange-AMR with $\mathcal{A}_{o2}$: $\tau = 0.2$, Lagrange-AMR with $\mathcal{A}_{\text{diff}}$: $\tau = 0.2$, Euler-AMR with $\mathcal{A}_{o2}$: $\tau = 0.12$, Euler-AMR with $\mathcal{A}_{\text{diff}}$: $\tau = 0.13$, pure Euler: $\tau = 7.4$. The higher runtime for the Lagrange-AMR cases is related to smaller timesteps associated with the smaller cells.

6.5. Triple point problem

We now consider a 3D extension of the 2D static triple point problem [50]. The mesh is divided into a high pressure region of 1 and a density of 1 left of $x = 2$, and assigned a material id 1 with $\gamma = 1.4$. The right region has a lower pressure of 0.1 and is divided in the transverse direction between a corner at 1 assigned material id 2 with $\gamma = 1.5$, and the rest is assigned material 1 with a density of 0.125. The box is a $7 \times 3 \times 3$ rectangle with symmetry boundary conditions.

First, we carry a set of simulations in 2D to illustrate how the refinement criteria captures the various discontinuities. The 2D Euler-AMR cases are initialized on a 35×16 mesh with a maximum refinement depth of 3. The criteria $\mathcal{A}_{o2}$ and $\mathcal{A}_{\text{diff}}$ are considered with a threshold T ranging from 0.02 to 0.20. The results for the density and the position of the material interface at $t = 0.5$ are shown in Fig. 30. All simulations give qualitatively similar results for the position of the discontinuities and morphology of the vortex. Once again, we find that the error-based criterion $\mathcal{A}_{o2}$ adds significantly fewer mesh cells. Interestingly, for threshold values of $T \geq 0.1$, we note that criterion $\mathcal{A}_{o2}$ loses resolution in some of the discontinuities. However, this is a value for T that we have determined to be too high from the Sod and interacting blast-wave test cases, such that this behavior is not unexpected.

Detailed results for the case $\mathcal{A}_{o2}$ with $T = 0.05$ are shown in Fig. 31, and illustrate how the various components in the criteria (partial volume fractions, density, pressure, velocity) act to refine different regions

of the mesh. We also show the dynamic mesh partitioning in panel (d), which illustrates how the load is kept balanced throughout the refinement process.

More detailed results for the Euler-AMR case with $\mathcal{A}_{\text{O}_2}$, $T = 0.05$ at $t=0.25$ and $t=0.5$ are shown in Fig. 31. These illustrate how the combination of criteria on several variables (density, partial volume fractions, velocities, pressure) captures the variety of discontinuities present in the calculation. We also show the dynamic mesh partitioning in panel (d), which illustrates how the load is kept balanced throughout the refinement process.

A second set of calculations is run in 3D. Similarly to the 2D case, the starting resolution for the AMR simulation is $35 \times 16 \times 16$, and the refinement depth is $l_{\text{max}} = 3$. The results using $\mathcal{A}_{\text{O}_2}$ and $T = 0.05$ are shown in Fig. 32, and are qualitatively similar to the 2D ones. The AMR result is compared to a pure Euler calculation performed on a mesh of $280 \times 128 \times 128$ in Fig. 33, and demonstrates good agreement in terms of the position of the material interface and the density profile. In that case, the AMR calculation was 3.3 times faster than the non-AMR simulation. While one could have expected a better speed-up, the runtime advantage of AMR will increase further with the refinement depth l_{max} . The number of cells in the mesh varies between ~ 3.5 and 7 times less than that of the pure Euler case, and is shown in Fig. 34.

7. Conclusion

We have presented a methodology to integrate a 3D forest of octree AMR framework with 3D cell-centered ALE hydrodynamics. This coupling requires two main components: the transcription of mesh connectivity between two different frameworks and adaptation of refinement/coarsening algorithms to the constraints of Lagrangian hydrodynamics. We summarize below our findings for each component.

The first component relates to the application of AMR mesh structures to unstructured Lagrangian hydrodynamics, which has posed two main challenges. First, the connectivity of AMR meshes is most often represented in a structured way, because the refinement of mesh cells introduces natural relationship between cells, and because it occurs simultaneously along the 3 basis directions. This is contrary to the unstructured Lagrangian framework, where there is no assumed organization of the mesh cells and their connectivity. The second difficulty stems from the fact that conventional AMR applications are often related to direct-Euler schemes, in which the cell nodes are not central to their formulation. Mostly, they are used as stencil for integration schemes, but do not have geometric meaning, contrary to the cell itself. As such, most AMR libraries, including *p4est*, do not ascribe physical meaning to nodes. Practically, this means that the node-based connectivity in these AMR libraries is lacking or absent, and that handling of nodes in MPI communications is also lacking or absent. In contrast, in the Lagrangian framework, nodes define the geometry of the cells, which in turn dictate its volume and energy/mass conservation properties through the Geometrical Conservation Law. Another singular distinction is that the hanging nodes in typical direct Euler-AMR frameworks are nodes that split edges or faces and require specific treatments, while in the unstructured Lagrangian framework they simply represent additional degrees of freedom for the polygonal cells and are not treated differently for the hydrodynamics. Finally, the nodal solvers of the 3D EUCCLHYD framework require synchronization to be carried out on nodal communicators, something that is not supported by *p4est* since it considers nodes as integration stencils. For all these reasons, significant work was required to convert from the topological information of a forest of octree framework to that of a polygonal mesh defined by its nodes. We have demonstrated that this conversion can be done practically and have presented some of the key algorithms involved.

The second component relates to the adaptation of conventional refinement and coarsening algorithms to the Lagrangian framework. Refinement and coarsening consists in splitting a cell into smaller ones or grouping cells into a larger one. In typical direct Euler-AMR approaches (excluding high order element approaches), this operation does not pose particular difficulties because the faces of mesh cells are flat. Notably, it is straightforward to ensure that mass is conserved during these operations because the sum of the volume of refined cells is trivially equal to that of the parent, and similarly for coarsening. We have demonstrated that this simple property is lost in the 3D Lagrangian framework because polygonal faces are not flat. Lagrangian schemes rely on various triangulations of cell faces in order to solve the Lagrangian hydrodynamics phase and ascribe a subset of the computational domain to the cell. In the particular case

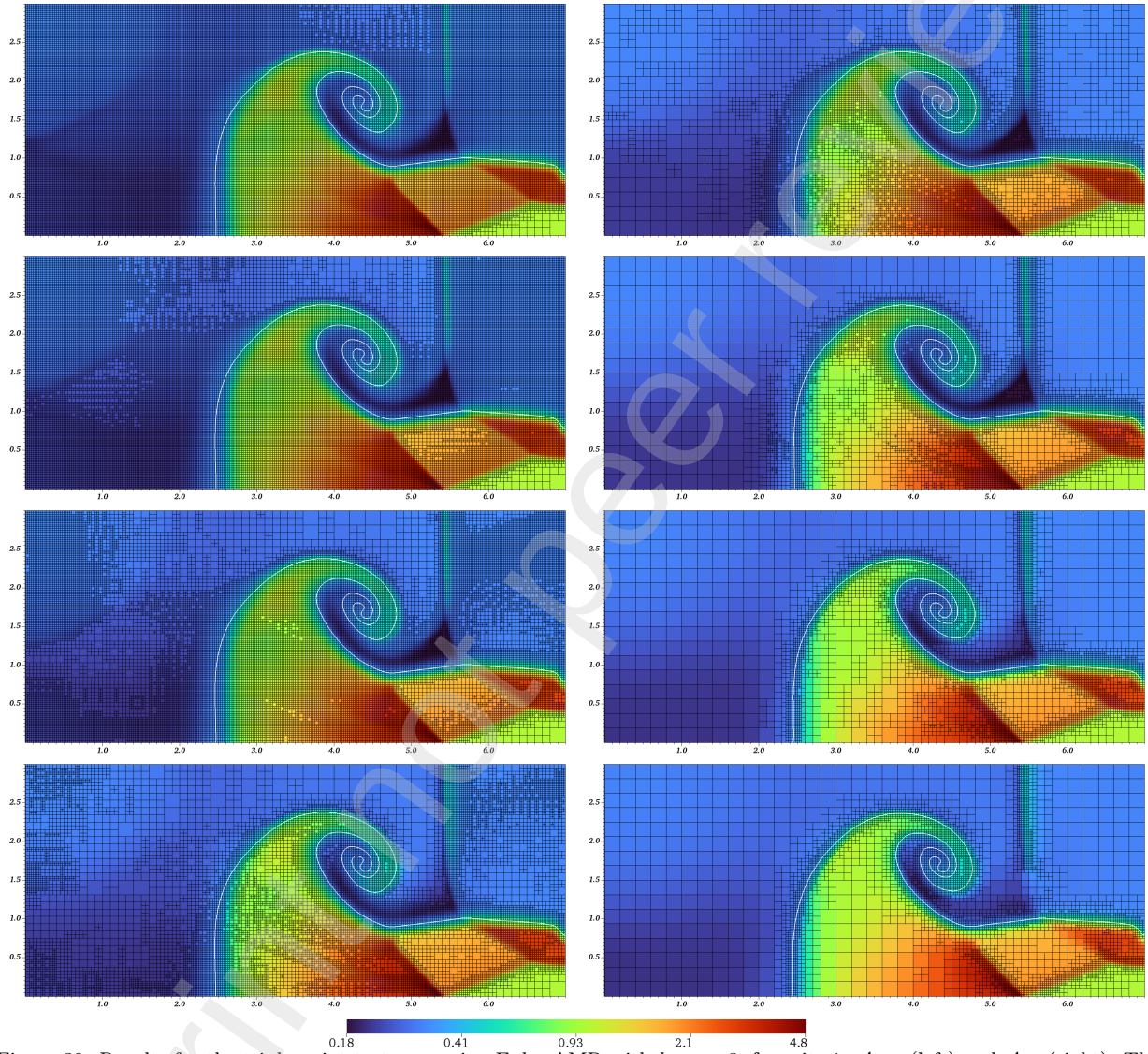


Figure 30: Results for the triple-point test case using Euler-AMR with $l_{\text{max}} = 3$, for criteria $\mathcal{A}_{\text{diff}}$ (left) and $\mathcal{A}_{\text{o2}}$ (right). The different rows use the threshold values of $T = 0.02$, $T = 0.05$, $T = 0.10$ and $T = 0.20$ (from top to bottom), respectively, and show the density on a log scale. The white contour shows the 50% partial volume fraction isocontour.

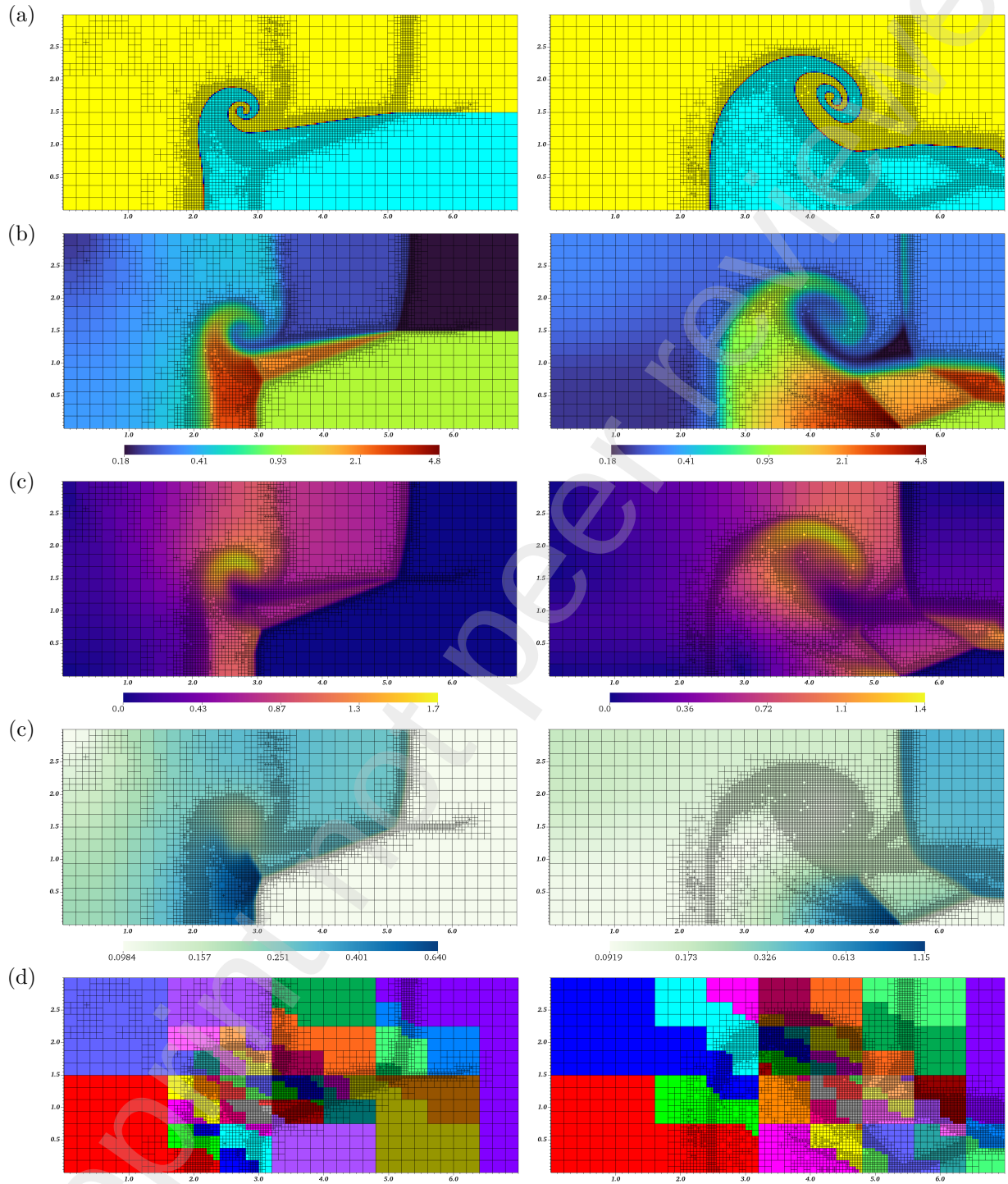


Figure 31: 2D simulation results for the triple-point test case at (left) $t=0.25$ and (right) $t=0.5$, for Euler-AMR with $\mathcal{A}_{o2}$, $T = 0.05$ and $l_{\max} = 3$. The panels show (from top to bottom): (a) partial volume fraction, (b) material density (log scale), (c) velocity magnitude, (d) material pressure (log scale) and (e) MPI domain decomposition (72 ranks in total).

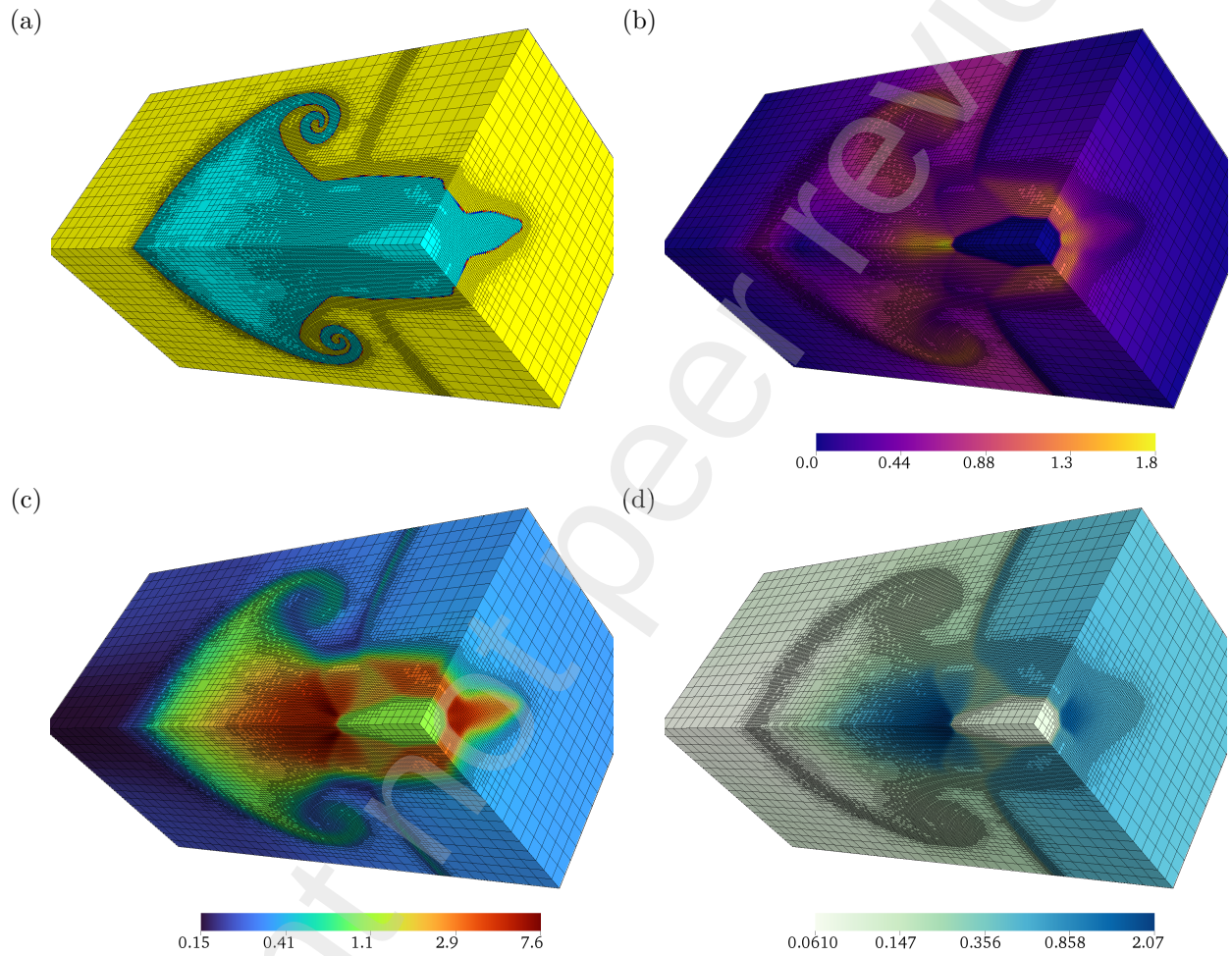


Figure 32: 3D simulation results for the triple-point test case at $t=0.5$ for the Euler-AMR case with $\mathcal{A}_{O2}$, $T = 0.05$ and $l_{\max} = 3$. The panels show: (a) partial volume fraction, (b) velocity magnitude, (c) material density (log scale) and (d) material pressure (log scale).

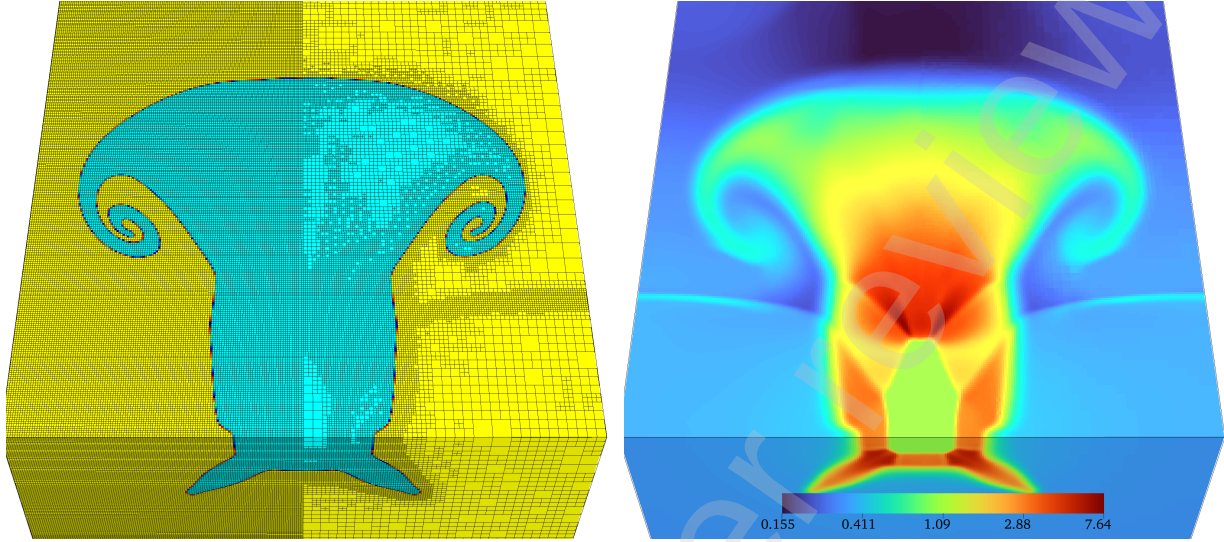


Figure 33: Comparison of the 3D triple-point simulation results between pure Euler and Euler-AMR, for (left) the partial volume fraction and (right) the material density in log scale. In each panel, the left half is the pure Euler result and the right half the Euler-AMR result. The AMR calculation uses $\mathcal{A}_{o2}$, $T = 0.05$ and $l_{\max} = 3$.

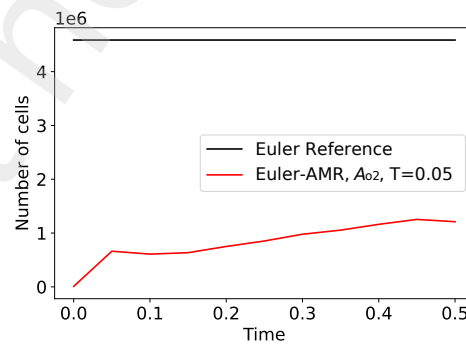


Figure 34: Comparison of the number of cells in the mesh as a function of time for the triple-point test case in 3D, between pure Euler (black) and Euler-AMR with $\mathcal{A}_{o2}$, $T = 0.05$ and $l_{\max} = 3$ (red). The AMR case is setup such that it matches the resolution of the non-AMR case at maximum refinement.

of EUCCLHYD, that triangulation is achieved by introducing a fictitious node at the barycenter of the face. When a face is refined into 4 sub-faces, each sub-face is also triangulated using a new fictitious node at its barycenter. The sum of the volumes incident on these 4 sub-faces, using their respective triangulations, is in general not equal to that computed on the original face. In order to resolve that volume discrepancy, we have given a method that consists in (i) adding the new degrees of freedom from refinement at adequate locations, and (ii) by preprocessing the mesh before refinement. Similar issues arise during coarsening, and we have also proposed a preprocessing method to ensure volume-conservative coarsening. In both cases, this preprocessing involves an intermediate ALE step, such that in our framework a pure Lagrangian-AMR method does not actually exist, and is really an ALE-AMR method. Finally, we have highlighted how these regridding operations carried out in the Lagrangian framework can naturally lead to tangled cells. For coarsening operations, this tangling occurs simply by virtue of the geometrical positioning of the nodes with respect to neighbor cells, while for refinement, it is related to preprocessing operations. This additional difficulty is significant, and is handled through a robust disentangling algorithm that is compatible with the nonconformal framework. That algorithm is related to our 3D ALE-AMR rezoning algorithm presented in part II of this work, in Ref. [13]. It relies on a weighted line-sweep method [51, 52] adapted to AMR grids and to block-structured meshes.

After having presented our Lagrange-AMR framework, we have proposed two refinement criteria to control regridding operations. The first is adapted from traditional direct Euler-AMR methods and relies on the computation of absolute differences in hydrodynamic quantities between a cell and its nodal neighbors. This direct-Euler inspired criterion is, in its principle, not fully relevant for Lagrangian methods where the regions of large gradients do not necessarily require refinement, a straightforward example being that of a shock. For that purpose, we propose a second criterion that is based on a local error estimate of the hydrodynamics scheme. More precisely, we compute the deviation between the second-order reconstruction of hydrodynamic quantities in a cell extrapolated to its neighbors with their actual values. This leads to refinements of cells only in regions where the local reconstructions deviate from second order. Our method is applied to a variety of test cases in order to demonstrate (i) the accuracy of the Lagrange-AMR and indirect Euler-AMR methods compared to pure Lagrange or pure indirect Euler, (ii) assess the relative merits and shortcomings of our proposed refinement criteria and set ranges on acceptable threshold values, and (iii) assess the runtime advantage of using the AMR framework. Overall, we find excellent performance of our ALE-AMR algorithm. Results across the various test cases are as accurate as their non-AMR versions, while offering speed-ups in the 3-5 range. Better speed-up is anticipated when increasing further the refinement depth and when considering additional physics modules that are more sensitive to the total number of cells in the mesh, such as diffusion solvers or particle tracing models.

This work has shown that, starting from a 3D indirect ALE code, the mesh topology, memory management, and load balancing requirements can be delegated to an external library (here the *p4est* library) to unlock ALE-AMR capabilities. As straightforward perspectives, one can quote the rezoning (regularization) mesh step, which should be carefully extended to this AMR framework. This point is the main subject of Part II. In addition, the multiscale simulation capabilities of the present numerical strategy open the way toward the simulations of complex physical problems. Consequently, in the future the physics applications side will be largely explored. The present technology will be applied to the simulation of a wide range of physical problems such as the ones encountered in the fields of inertial confinement fusion, materials dynamic behavior or atmospheric reentry.

8. Acknowledgment

This work is supported by the U.S. Department of Energy (DOE), Office of Science, Fusion Energy Sciences, under Award No. DE-SC0024863: IFE-STAR and AWD-00007026: INFUSE.

References

- [1] V. Maltsev, M. Skote, and P. Tsoutsanis. High-order methods for diffuse-interface models in compressible multi-medium flows: A review. *Physics of Fluids*, 34(2):021301, 2022.

- [2] R. Saurel and C. Pantano. Diffuse-interface capturing methods for compressible two-phase flows. *Annual Review of Fluid Mechanics*, 50:105–130, 2018.
- [3] S. Kokh and F. Lagoutière. An anti-diffusive numerical scheme for the simulation of interfaces between compressible fluids by means of a five-equation model. *J. Comput. Phys.*, 229(8):2773–2809, 2010.
- [4] J. Breil. *Numerical methods for Lagrangian and Arbitrary-Lagrangian-Eulerian Hydrodynamic Contribution to the simulation of High-Energy-Density-Physics Problems*. Habilitation à diriger des recherches, Bordeaux University, 2016.
- [5] B. Manach-Pérennou, R. Chauvin, S. Guisset, and A. Llor. Cell-centered lagrangian scheme for multi-material flows with pressure equilibration. *Comput. & Fluids*, 250:105705, 2023.
- [6] D.J. Benson. Computational methods in Lagrangian and Eulerian hydrocodes. *Comput. Methods Appl. Mech. Eng.*, 99:235–394, 1992.
- [7] J. von Neumann and R.D. Richtmyer. A method for the numerical calculations of hydrodynamical shocks. *J. Appl. Phys.*, 21:232–238, 1950.
- [8] M.L. Wilkins. Calculation of elastic plastic flow. *Methods Comput. Phys.*, 3:211–263, 1964.
- [9] S.K. Godunov, A. Zabrodine, M. Ivanov, A. Kraiko, and G. Prokopov. Résolution numérique des problèmes multi-dimensionnels de la dynamique des gaz. *Éditions Mir Moscou*, 1979.
- [10] G. Scovazzi, J.N. Shadid, E. Love, and W.J. Rider. A conservative nodal variational multiscale method for Lagrangian shock hydrodynamics. *Comput. Methods Appl. Mech. Eng.*, 199:3059–3100, 2010.
- [11] V.A. Dobrev, T.E. Ellis, T.V. Kolev, and R.N. Rieben. High-order curvilinear finite elements for axisymmetric Lagrangian hydrodynamics. *Comput. and Fluids*, 83:58–69, 2013.
- [12] P. Knupp, L.G. Margolin, and M.J. Shashkov. Reference Jacobian optimization-based rezone strategies for Arbitrary Lagrangian-Eulerian methods. *J. Comput. Phys.*, 176:93–128, 2002.
- [13] A. Colaitis, S. Guisset, and J. Breil. An AMR cell-centered scheme for 3D multi-material ALE hydrodynamics, Part II: line-sweep ALE rezoning for non-conformal block-structured AMR meshes. *In preparation*, 2025.
- [14] L.G. Margolin and M. Shashkov. Second-order sign-preserving remapping on general grids. *J. Comput. Phys.*, 184:266–298, 2003.
- [15] M. Berger and J. Oliger. Adaptive mesh refinement for hyperbolic partial differential equations. *J. Comput. Phys.*, 53(3):484–512, 1984.
- [16] J. Bell, M. Berger, J. Saltzman, and M. Welcome. Three-dimensional adaptive mesh refinement for hyperbolic conservation laws. *SIAM Journal on Scientific Computing*, 15(1):127–138, 1994.
- [17] M.J. Berger and P. Colella. Local adaptive mesh refinement for shock hydrodynamics. *J. Comput. Phys.*, 82(1):64–84, 1989.
- [18] R. Pember, J. Bell, P. Colella, W. Curtchfield, and M. Welcome. An adaptive cartesian grid method for unsteady compressible flow in irregular regions. *J. Comput. Phys.*, 120(2):278–304, 1995.
- [19] Kolja Brix, Silvia Melian, Siegfried Müller, and Gero Schieffer. Parallelisation of multiscale-based grid adaptation using space-filling curves. *Esaim: Proceedings*, 29:108–129, 12 2009.
- [20] A. Dubey, A. Almgren, J. Bell, M. Berzins, S. Brandt, G. Bryan, P. Colella, D. Graves, M. Lijewski, F. Löffler, B. O’Shea, E. Schnetter, B. Van Straalen, and K. Weide. A survey of high level frameworks in block-structured adaptive mesh refinement packages. *Journal of Parallel and Distributed Computing*, 74(12):3217–3227, 2014. Domain-Specific Languages and High-Level Frameworks for High-Performance Computing.
- [21] Carsten Burstedde, Lucas C. Wilcox, and Omar Ghattas. **p4est**: Scalable algorithms for parallel adaptive mesh refinement on forests of octrees. *SIAM Journal on Scientific Computing*, 33(3):1103–1133, 2011.
- [22] M. Kronbichler, T. Heister, and W. Bangerth. High accuracy mantle convection simulation through modern numerical methods. *Geophysical Journal International*, 191(1):12–29, 2012.
- [23] D. Arndt, W. Bangerth, D. Davydov, T. Heister, L. Heltai, M. Kronbichler, M. Maier, J.P. Pelteret, B. Turcksin, and D. Wells. The deal.ii finite element library: Design, features, and insights. *Comput. Methods Appl. Mech. Eng.*, 81:407–422, 2021.
- [24] F. Drui, A. Fikl, P. Kestener, S. Kokh, A. Larat, V. Le Chenadec, and M. Massot. Experimenting with the p4est library for amr simulations of two-phase flows. *ESAIM: Proceedings and Surveys*, 53:232–247, 03 2016.
- [25] R.W. Anderson, N.S. Elliott, and R.B. Pember. An arbitrary lagrangian-eulerian method with adaptive mesh refinement for the solution of the euler equations. *J. Comput. Phys.*, 199(2):598–617, 2004.
- [26] B S Ryujin. Performance and portability in the ares multi-physics code. 8 2015.
- [27] J. Baiges, R. Codina, A. Pont, and E. Castillo. An adaptive fixed-mesh ale method for free surface flows. *Comput. Methods Appl. Mech. Eng.*, 313:159–188, 2017.
- [28] J. Waltz and J. Bakosi. A coupled ale-amr method for shock hydrodynamics. *Comput. & Fluids*, 167:359–371, 2018.
- [29] Gabriel Georges, Jérôme Breil, and Pierre-Henri Maire. A 3D GCL Compatible Cell-Centered Lagrangian Scheme for Solving Gas Dynamics Equations. *J. Comput. Phys.*, 305(C):921–941, jan 2016.
- [30] G. Georges. *Développement d’un schéma aux volumes finis centré lagrangien pour la résolution 3D des équations de l’hydrodynamique et de l’hyperélasticité*. PhD thesis, Université de Bordeaux, 2016.
- [31] Bram van Leer. Towards the ultimate conservative difference scheme. ii. monotonicity and conservation combined in a second-order scheme. *J. Comput. Phys.*, 14(4):361–370, 1974.
- [32] B. Van Leer. Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov’s method. *J. Comput. Phys.*, 32(1):101–136, 1979.
- [33] D. Youngs. Time-dependent multi-material flow with large fluid distortion. *Numerical Methods in Fluid Dynamics*, 1982.
- [34] S. Galera, P.-H. Maire, and J. Breil. A two-dimensional unstructured cell-centered multi-material ale scheme using vof interface reconstruction. *J. Comp. Phys.*, 229(16):5755–5787, 2010.

- [35] J. Velechovsky, E. Kikinzon, N. Ray, H. Rakotoarivelo, A. Herring, M. Kenamond, K. Lipnikov, M. Shashkov, R. Garimella, and D. Shevitz. Multi-material swept face remapping on polyhedral meshes. *J. Comput. Phys.*, 469:111553, 2022.
- [36] B. Mirtich. Fast and accurate computation of polyhedral mass properties. *Journal of Graphics Tools*, 1(2):31–50, 1996.
- [37] T. Barth and D. Jespersen. The design and application of upwind schemes on unstructured meshes. *27th Aerospace Sciences Meeting*, 1989, 1989.
- [38] P.-H. Maire. A high-order one-step sub-cell force-based discretization for cell-centered lagrangian hydrodynamics on polygonal grids. *Comput. & Fluids*, 46(1):341–347, 2011. 10th ICFD Conference Series on Numerical Methods for Fluid Dynamics (ICFD 2010).
- [39] J. Velechovsky, J. Breil, and R. Liska. Flux corrected remapping using piecewise parabolic reconstruction for 2d cell-centered ale methods. *Int. J. Numer. Meth. Fluids*, 76(9):575–586, 2014.
- [40] S. Guisset, S. Peluchon, A. Colaïtis, and J. Breil. A 3d multi-material cell-centered indirect arbitrary lagrangian-eulerian scheme using compressive limiters. *Submitted*, 2024.
- [41] C. Chevalier and F. Pellegrini. Pt-scotch: A tool for efficient parallel graph ordering. *Parallel Computing*, 34(6):318–331, 2008. Parallel Matrix Algorithms and Applications.
- [42] K.T. Mandli, A.J. Ahmadi, M. Berger, D. Calhoun, D.L. George, Y. Hadjimichael, D.I. Ketcheson, G.I. Lemoine, and R.J. LeVeque. Clawpack: building an open source ecosystem for solving hyperbolic PDEs. *PeerJ Computer Science*, 2:e68, 2016.
- [43] W.F. Noh. Errors for calculations of strong shocks using an artificial viscosity and an artificial heat flux. *J. Comp. Phys.*, 72(1):78–120, 1987.
- [44] P. Woodward and P. Colella. The numerical simulation of two-dimensional fluid flow with strong shocks. *J. Comp. Phys.*, 54(1):115–173, 1984.
- [45] R.W. Anderson, N.S. Elliott, and R.B. Pember. An arbitrary lagrangian–eulerian method with adaptive mesh refinement for the solution of the euler equations. *J. Comp. Phys.*, 199(2):598–617, 2004.
- [46] Clawpack Development Team. Clawpack software, 2024. Version 5.10.0.
- [47] J.M. Stone, K.T., C.J. White, and K.G. Felker. The Athena++ Adaptive Mesh Refinement Framework: Design and Magnetohydrodynamic Solvers. *The Astrophysical Journal Supplement Series*, 249(1):4, 2020.
- [48] L. I. Sedov. *Similarity and Dimensional Methods in Mechanics*. Academic Press: New York, 1959.
- [49] J. R. Kamm and F. X. Timmes. On efficient generation of numerically robust Sedov solutions. *Technical Report LA-UR-07-2849*, Los Alamos National Laboratory, 2007.
- [50] M. Kucharik, R.V. Garimella, S.P. Schofield, and M.J. Shashkov. A comparative study of interface reconstruction methods for multi-material ALE simulations. *J. Comp. Phys.*, 229(7):2432–2452, 2010.
- [51] Jin Yao and Douglas Stillman. An equal-space algorithm for block-mesh improvement. *Procedia Engineering*, 163:199–211, 2016. 25th International Meshing Roundtable.
- [52] J. Breil, G. Damour, S. Guisset, and A. Colaïtis. 3d mesh regularization within an ale code using a weighted line sweeping method. *Submitted for publication*.